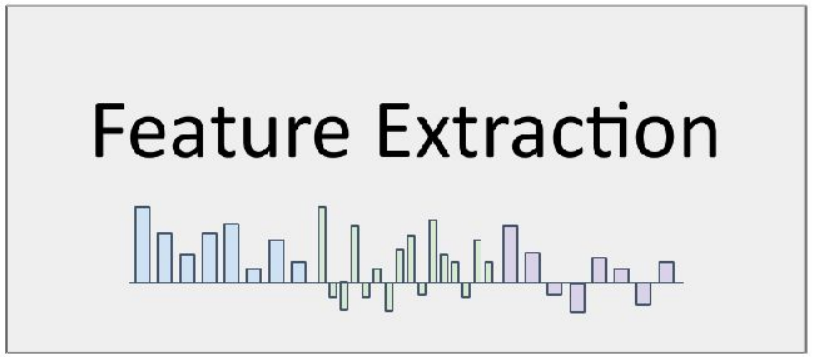
Lecture 5: Neural Networks

Part 2: Nonlinear Representations and Capacity

Image Features vs Neural Networks



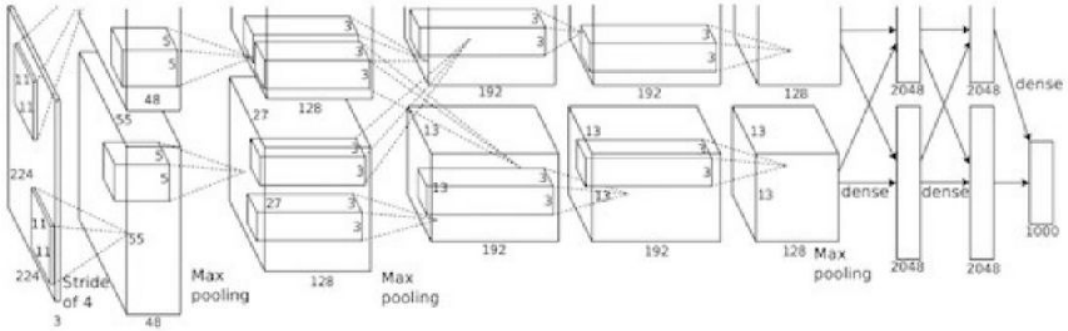


f

One score per class

training





Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012. Figure copyright Krizhevsky, Sutskever, and Hinton, 2012. Reproduced with permission.

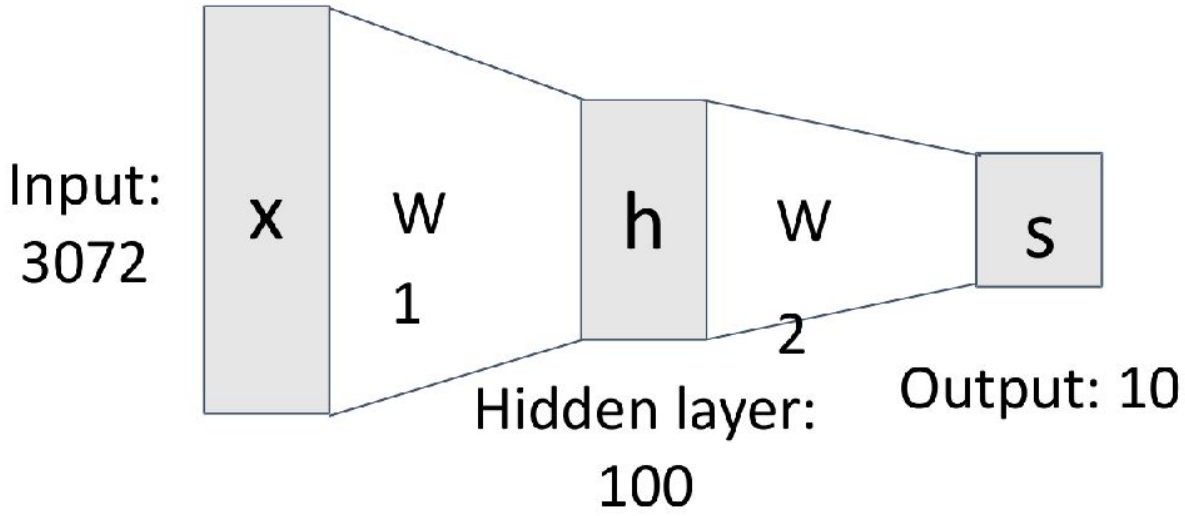
One score per class

training

Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

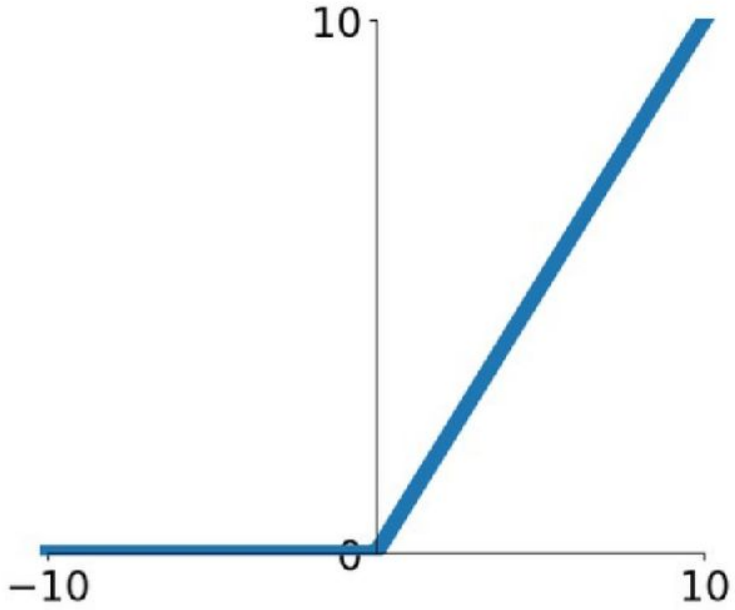
Activation Functions

2-layer Neural Network

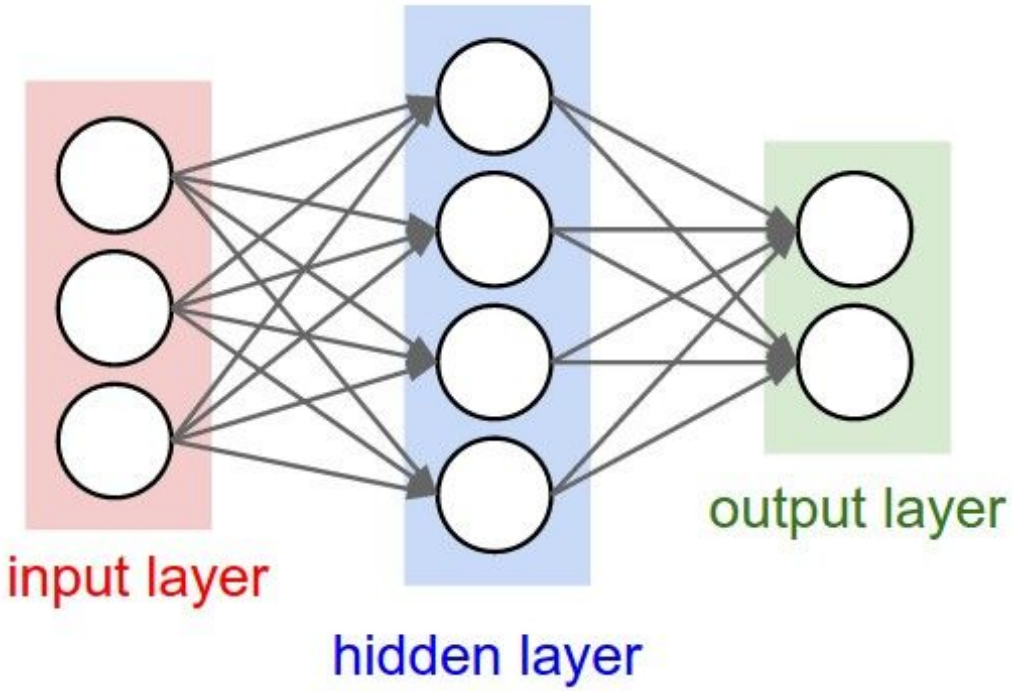
$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called “Rectified Linear Unit”

This is called the **activation function** of the neural network



One training step in PyTorch




```
import torch
from torch import nn

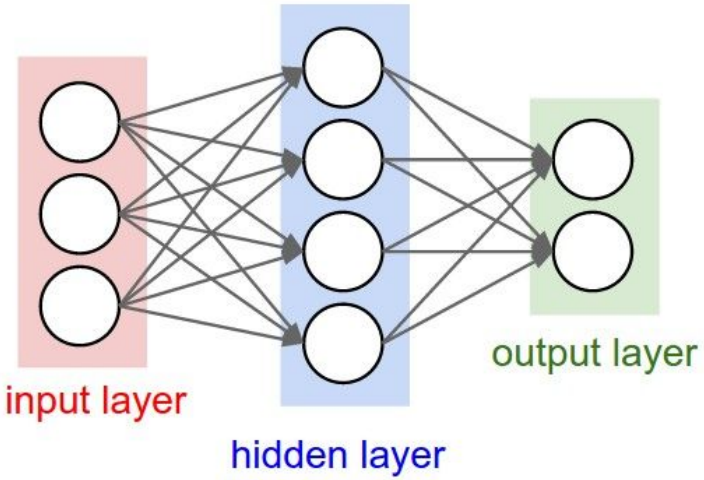
model = nn.Sequential(
   nn.Linear(3072, 100),
   nn.ReLU(),
   nn.Linear(100, 10),
)
optimizer = torch.optim.SGD(
   model.parameters(), lr=1e-2
)

scores = model(x)
loss = nn.functional.cross_entropy(scores, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

x has shape (N, 3072); y contains N class labels.

One training step in PyTorch



1

Define the model

```
import torch
from torch import nn

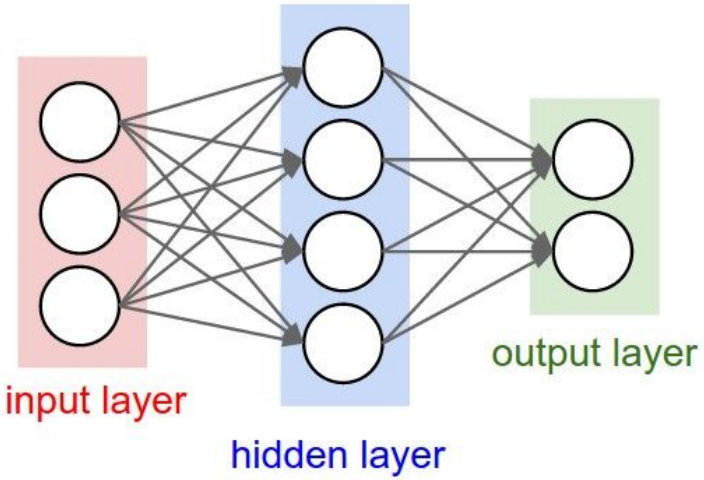
model = nn.Sequential(
    nn.Linear(3072, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
)
optimizer = torch.optim.SGD(
    model.parameters(), lr=1e-2
)

scores = model(x)
loss = nn.functional.cross_entropy(scores, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

x has shape (N, 3072); y contains N class labels.

One training step in PyTorch



2

Forward pass and loss

```
import torch
from torch import nn

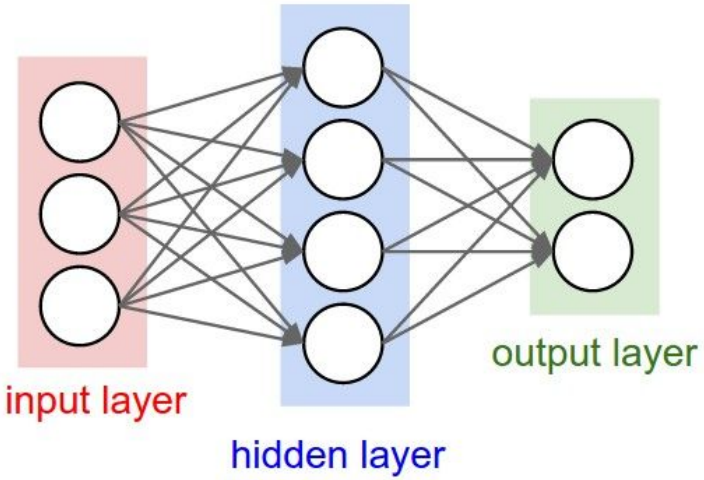
model = nn.Sequential(
    nn.Linear(3072, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
)
optimizer = torch.optim.SGD(
    model.parameters(), lr=1e-2
)

scores = model(x)
loss = nn.functional.cross_entropy(scores, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

x has shape (N, 3072); y contains N class labels.

One training step in PyTorch



3

Compute gradients

```
import torch
from torch import nn

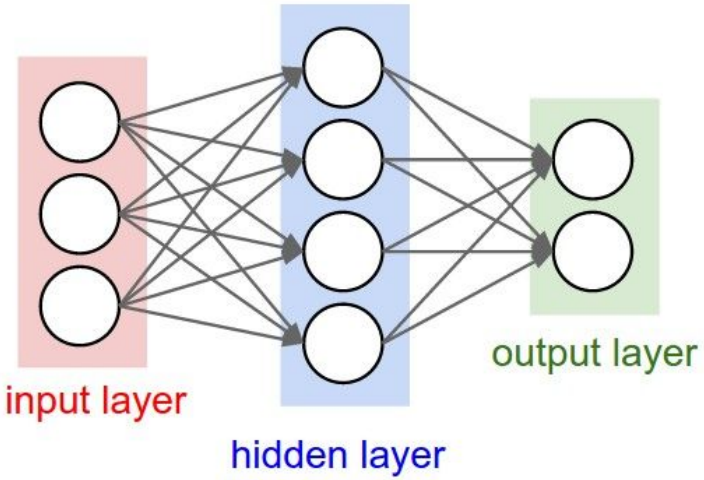
model = nn.Sequential(
    nn.Linear(3072, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
)
optimizer = torch.optim.SGD(
    model.parameters(), lr=1e-2
)

scores = model(x)
loss = nn.functional.cross_entropy(scores, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

x has shape (N, 3072); y contains N class labels.

One training step in PyTorch



4

Update parameters

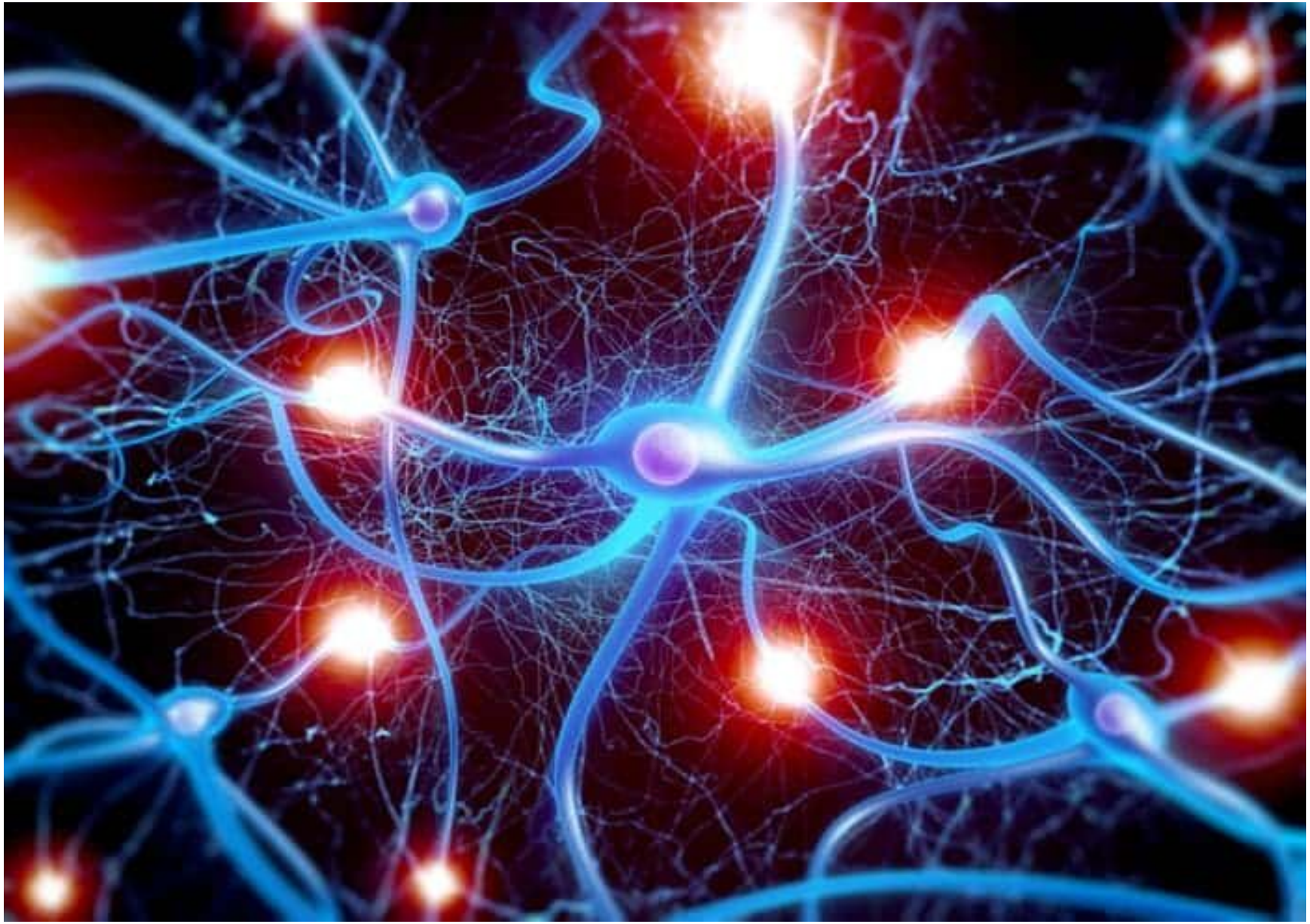
```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(3072, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
)
optimizer = torch.optim.SGD(
    model.parameters(), lr=1e-2
)

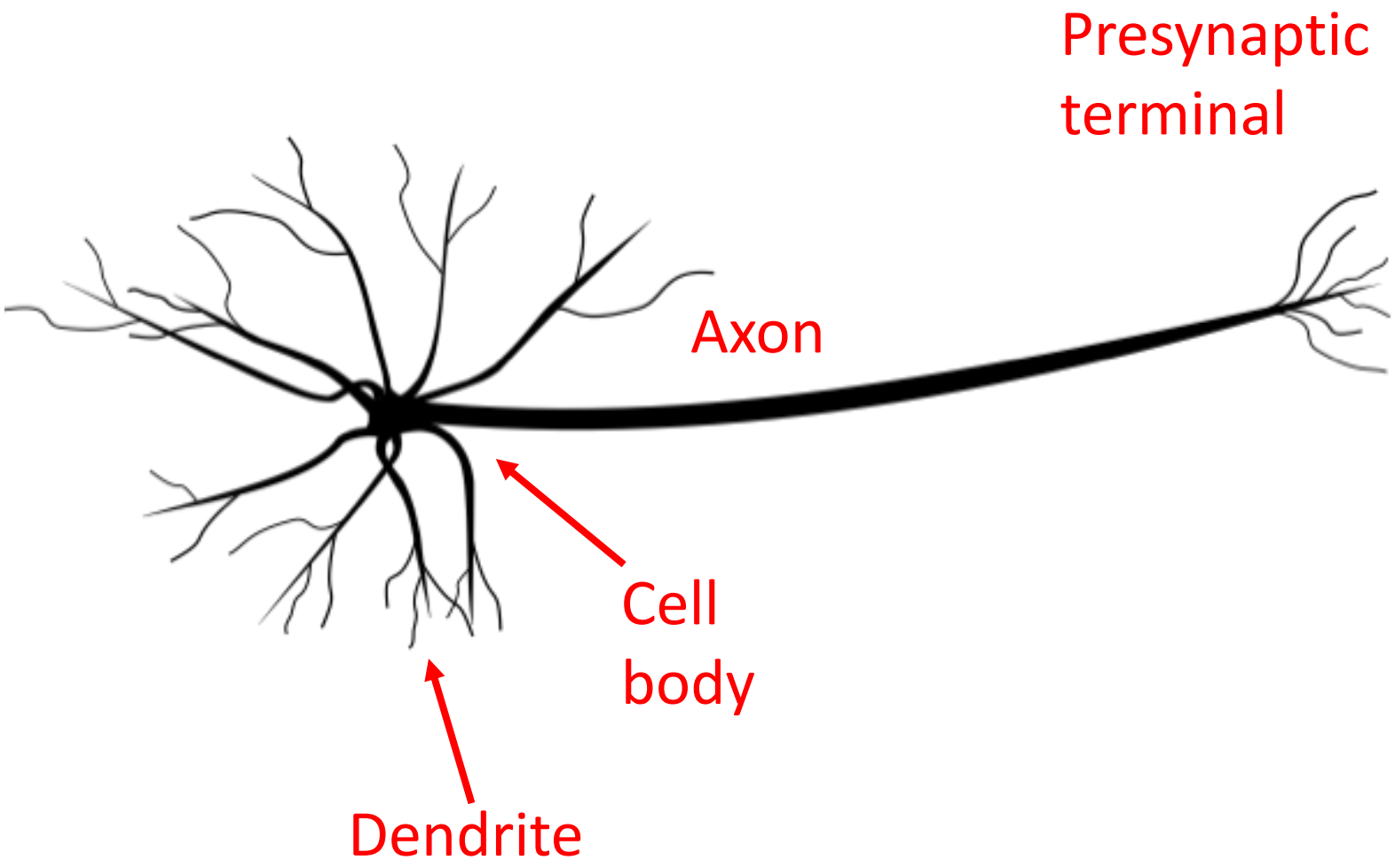
scores = model(x)
loss = nn.functional.cross_entropy(scores, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

x has shape (N, 3072); y contains N class labels.

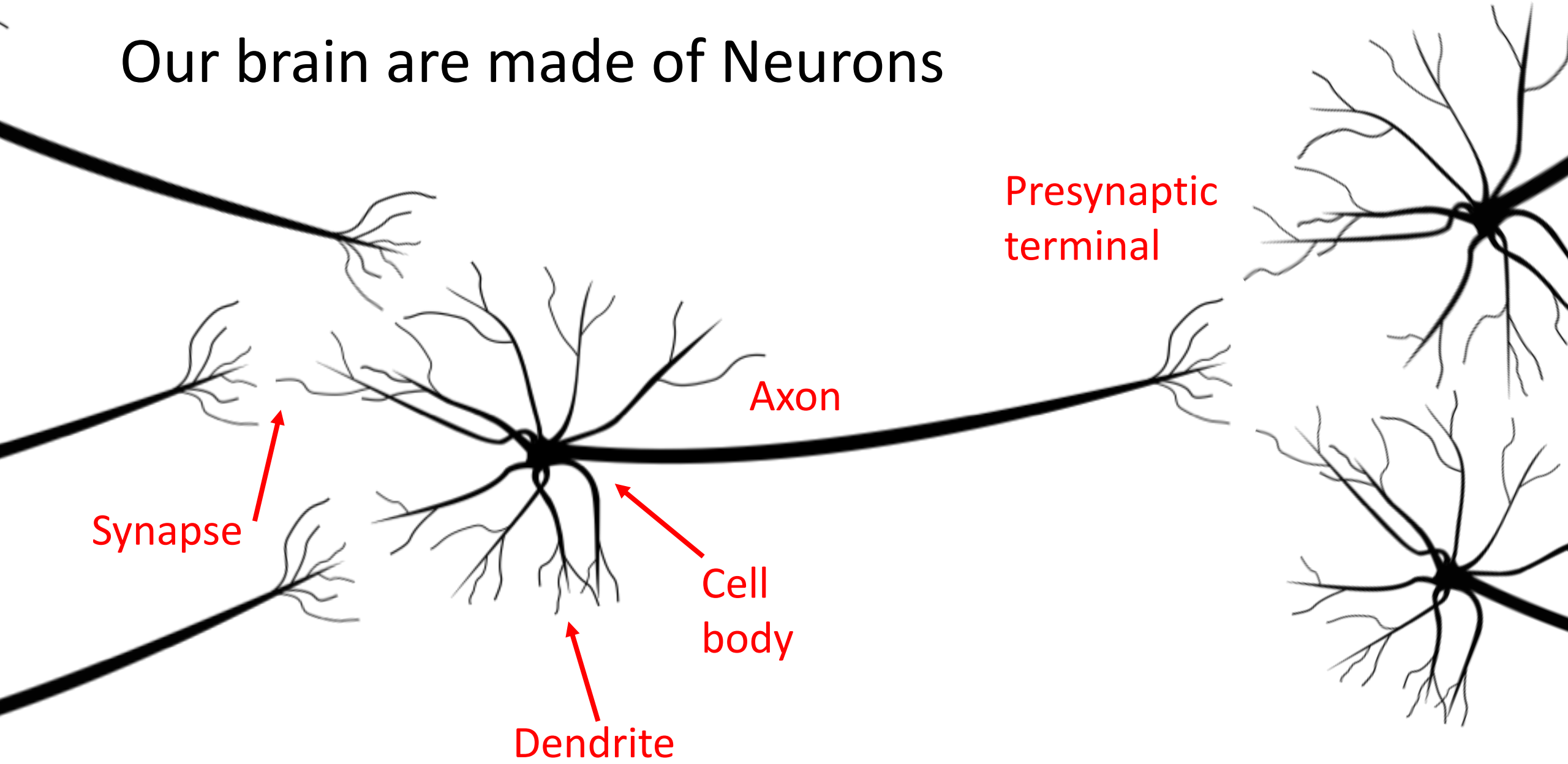


Our brain are made of Neurons



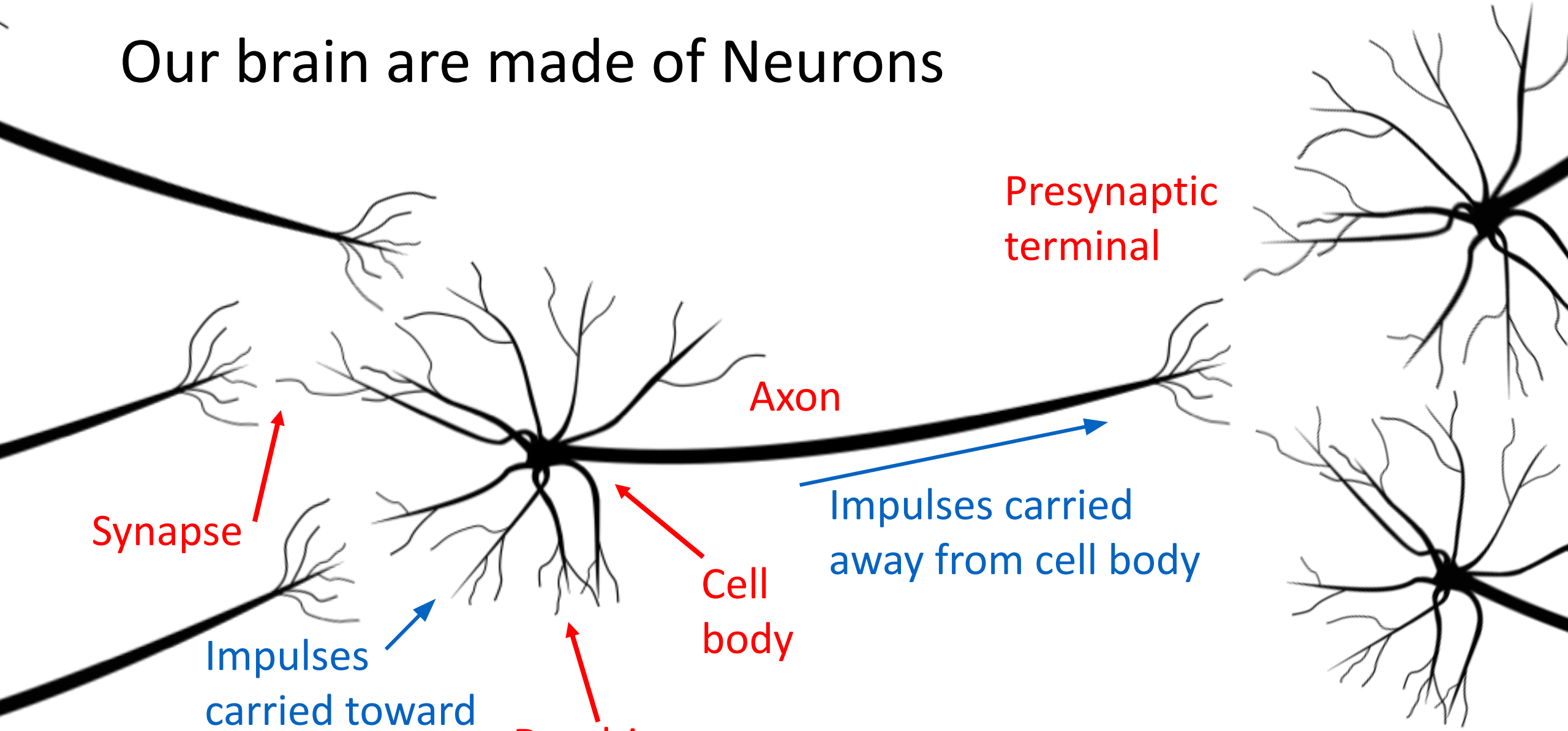
Neuron image by Felipe Perucho is licensed under CC-BY 3.0

Our brain are made of Neurons



Neuron image by Felipe Perucho is licensed under CC-BY 3.0

Our brain are made of Neurons



Neuron image by Felipe Perucho is licensed under CC-BY 3.0

Our brain are made of Neurons

Presynaptic terminal

Axon

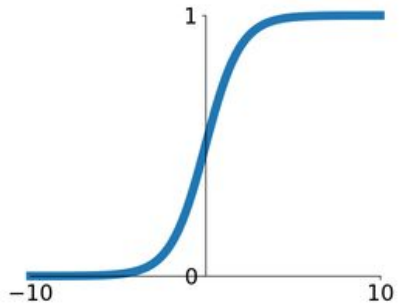
Synapse

Impulses carried away from cell body

Cell body

Impulses carried toward cell body

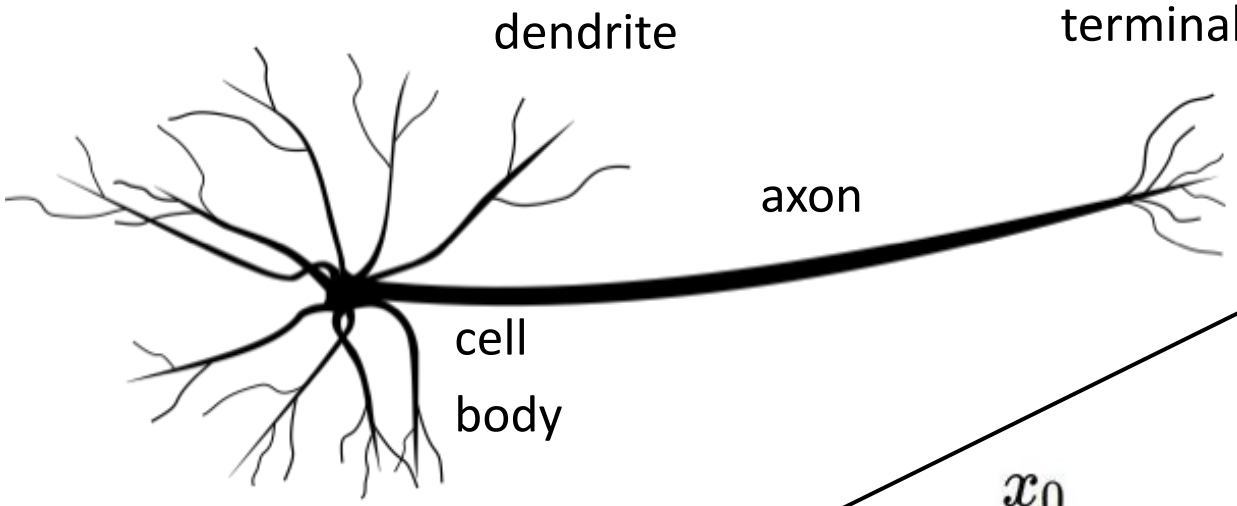
Dendrite



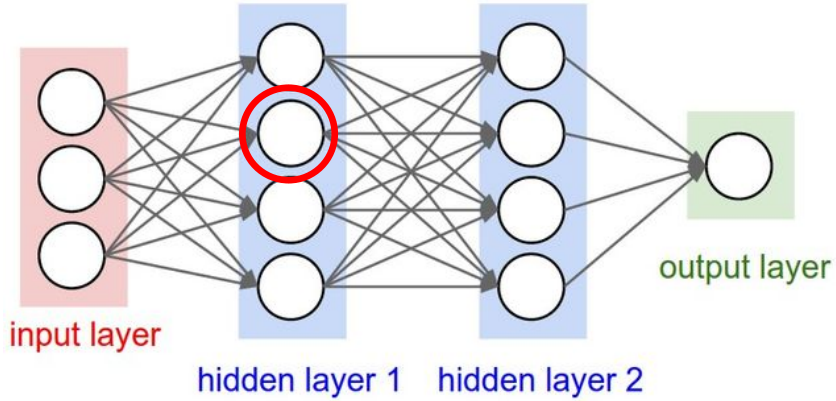
Firing rate is a nonlinear function of inputs

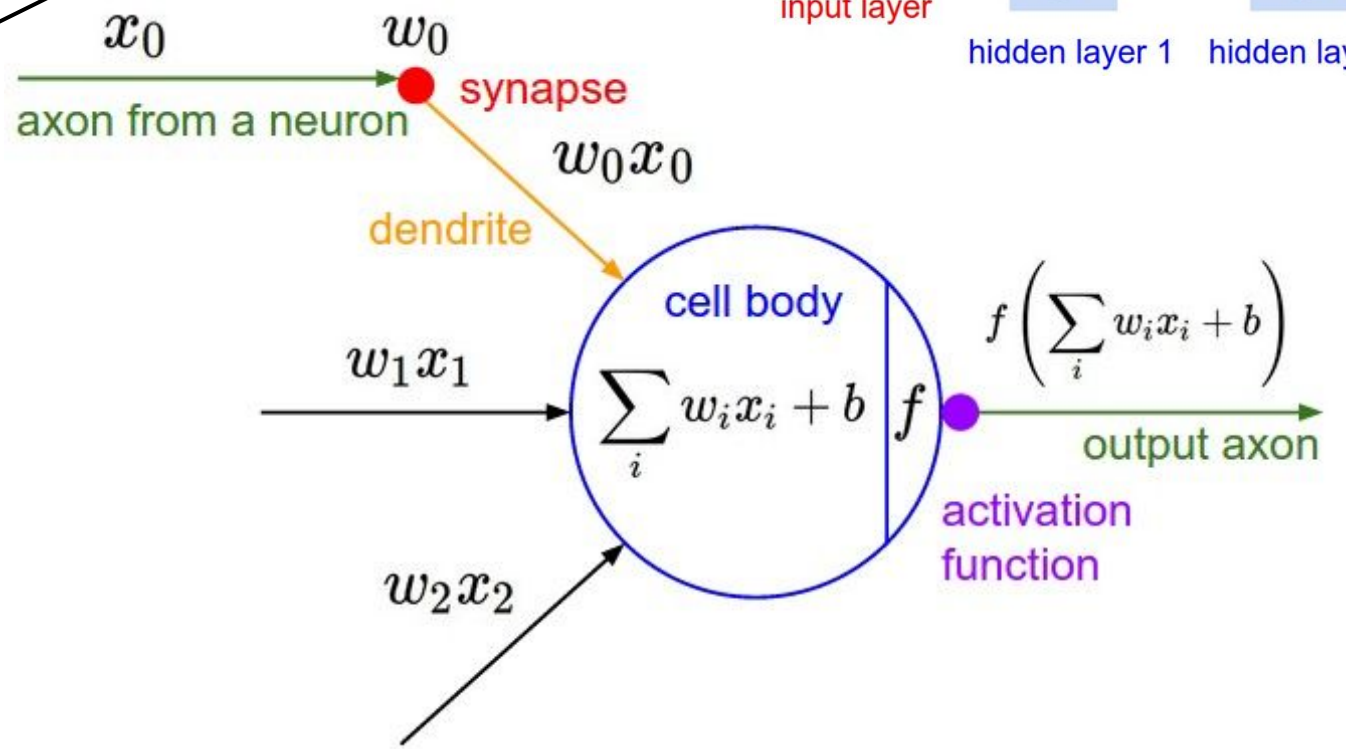
Neuron image by Felipe Perucho is licensed under CC-BY 3.0

Biological Neuron



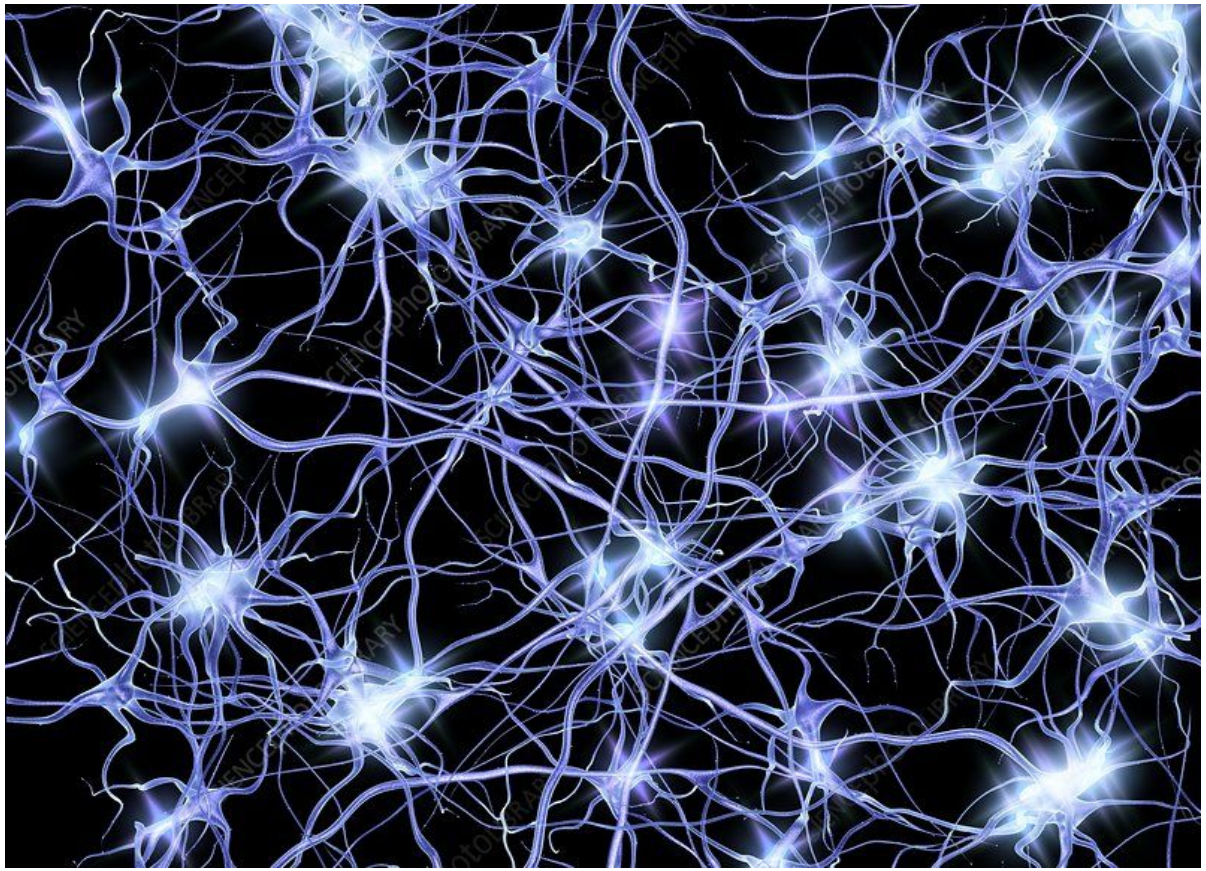
Artificial Neuron



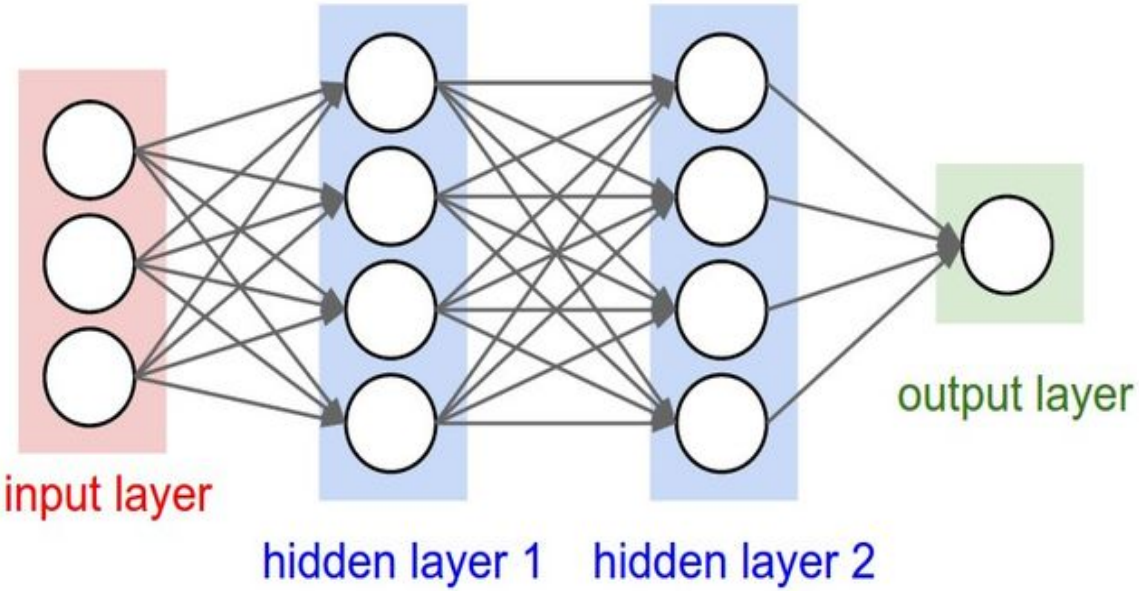


Neuron image by Felipe Perucho is licensed under CC-BY 3.0

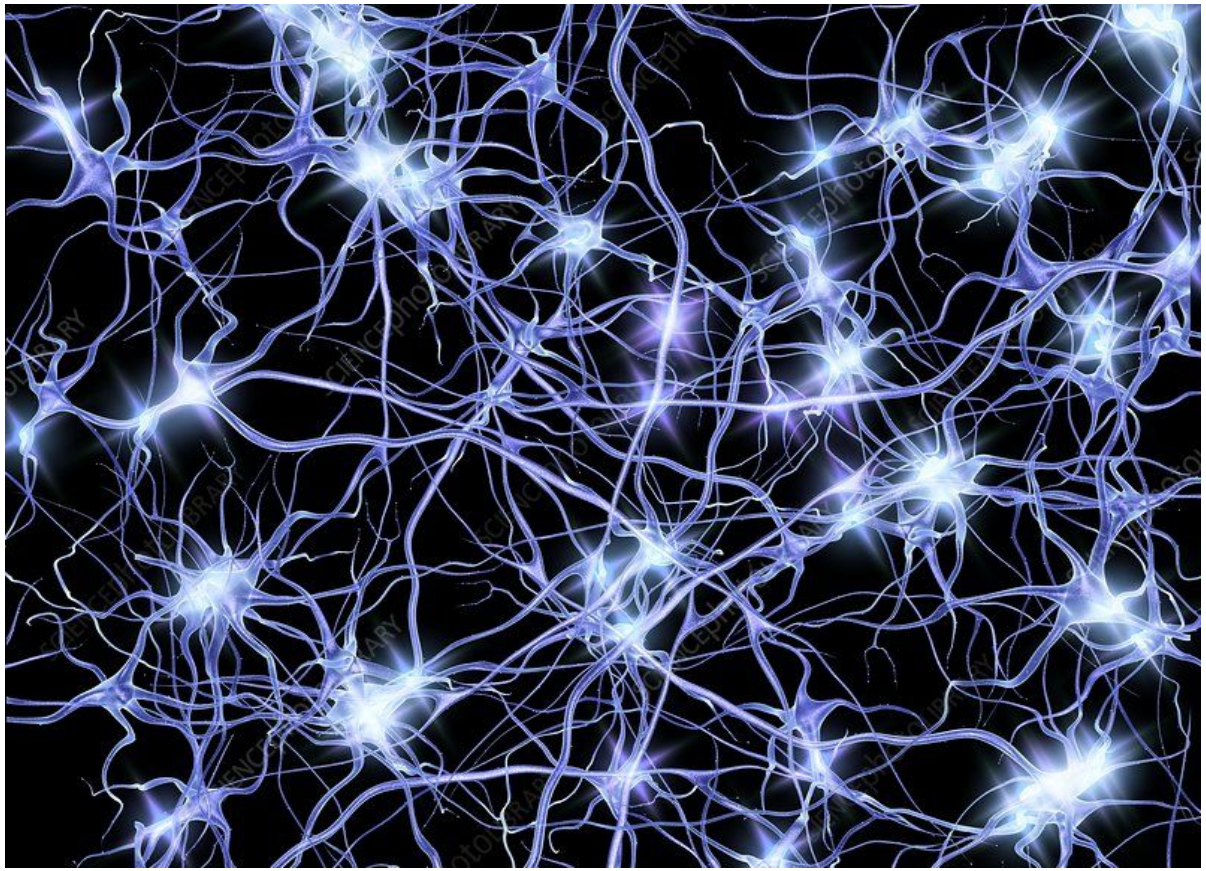
Biological Neuron:
Complex connectivity patterns



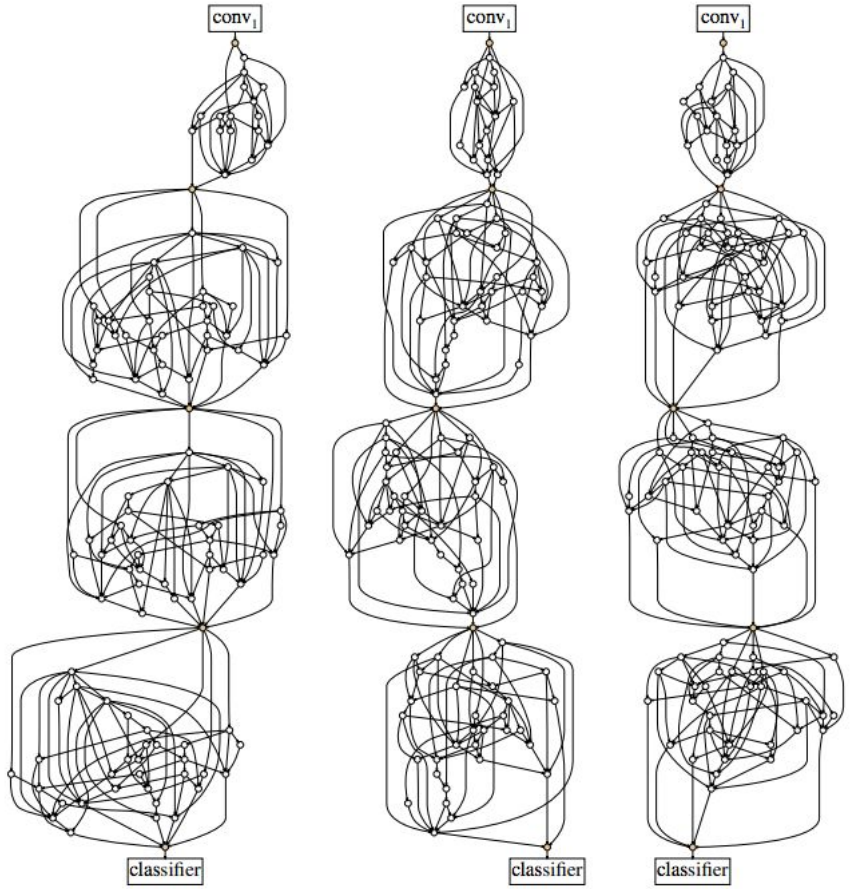
Neurons in a neural network:
Organized into regular layers for computational efficiency



Biological Neuron:
Complex connectivity patterns



But neural networks with random connections can work too!



Xie et al, “Exploring Randomly Wired Neural Networks for Image Recognition”, ICCV 2019

Limits of the biological analogy

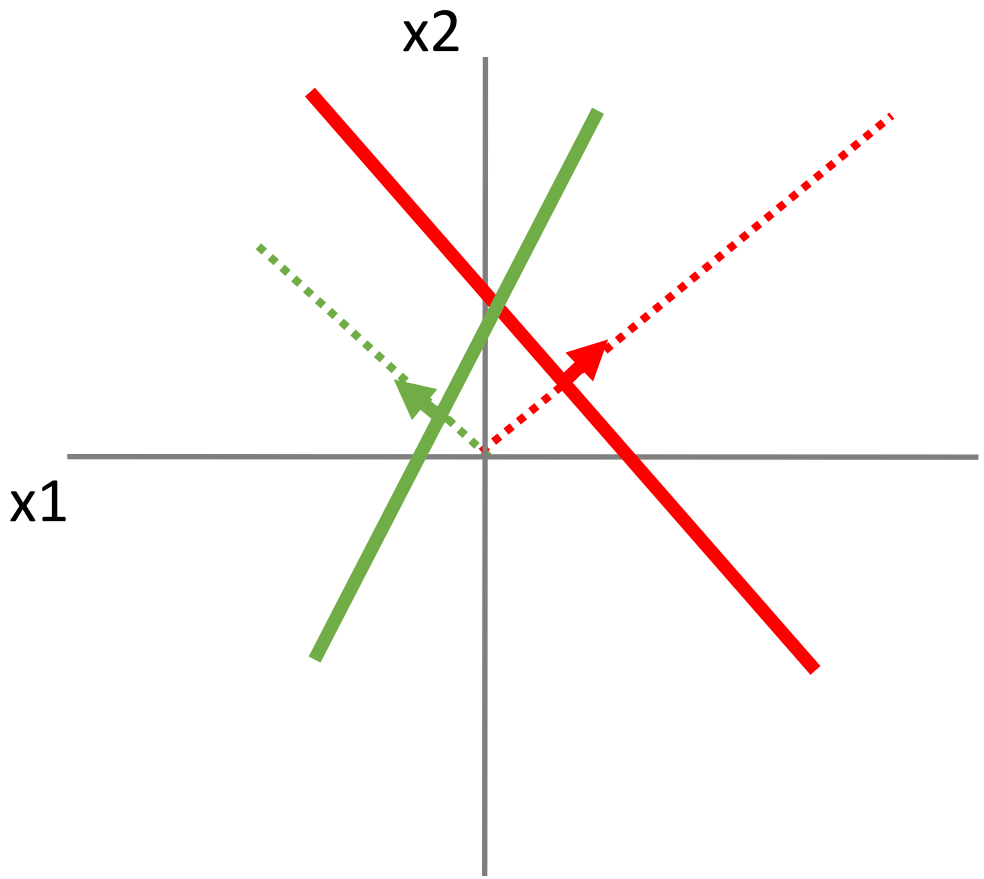
- Dendrites and synapses perform complex nonlinear, dynamic computations.
- A single scalar weight is only a loose analogy for a biological synapse.
- Artificial networks are organized for efficient computation, not biological realism.

London and Häusser, “Dendritic Computation”

Space Warping

Consider a linear transform: h = Wx

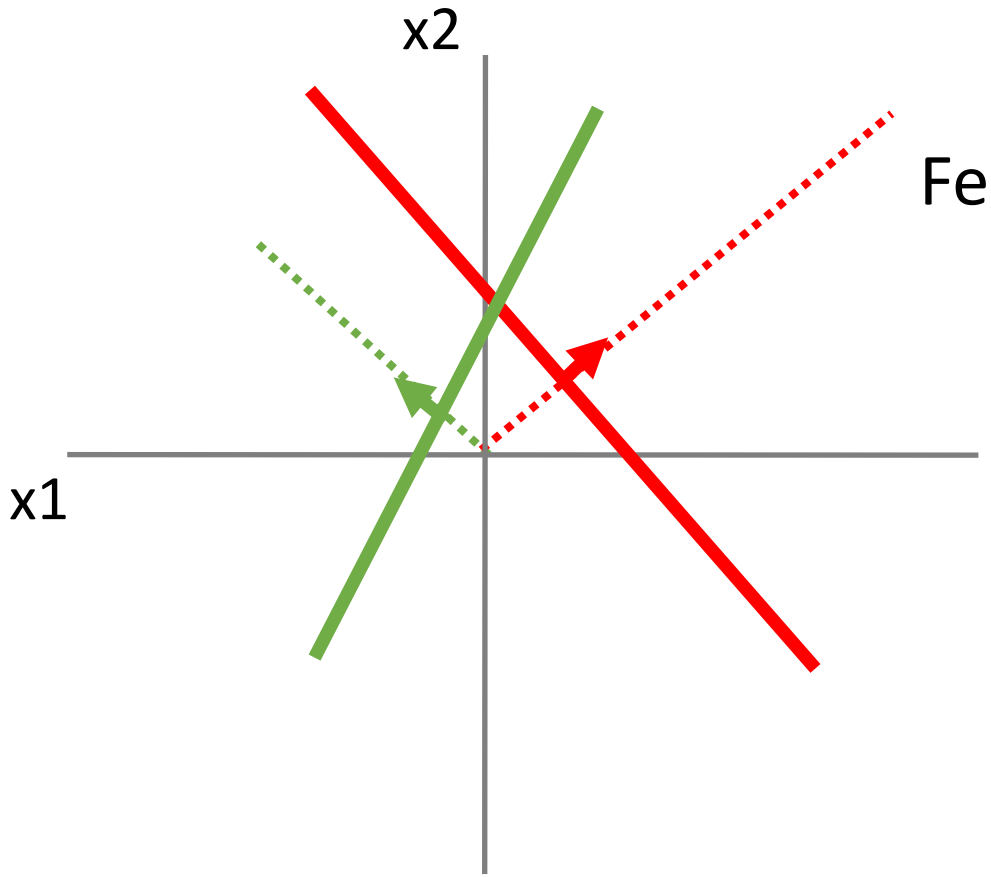
Where x, h are both 2-dimensional



Space Warping

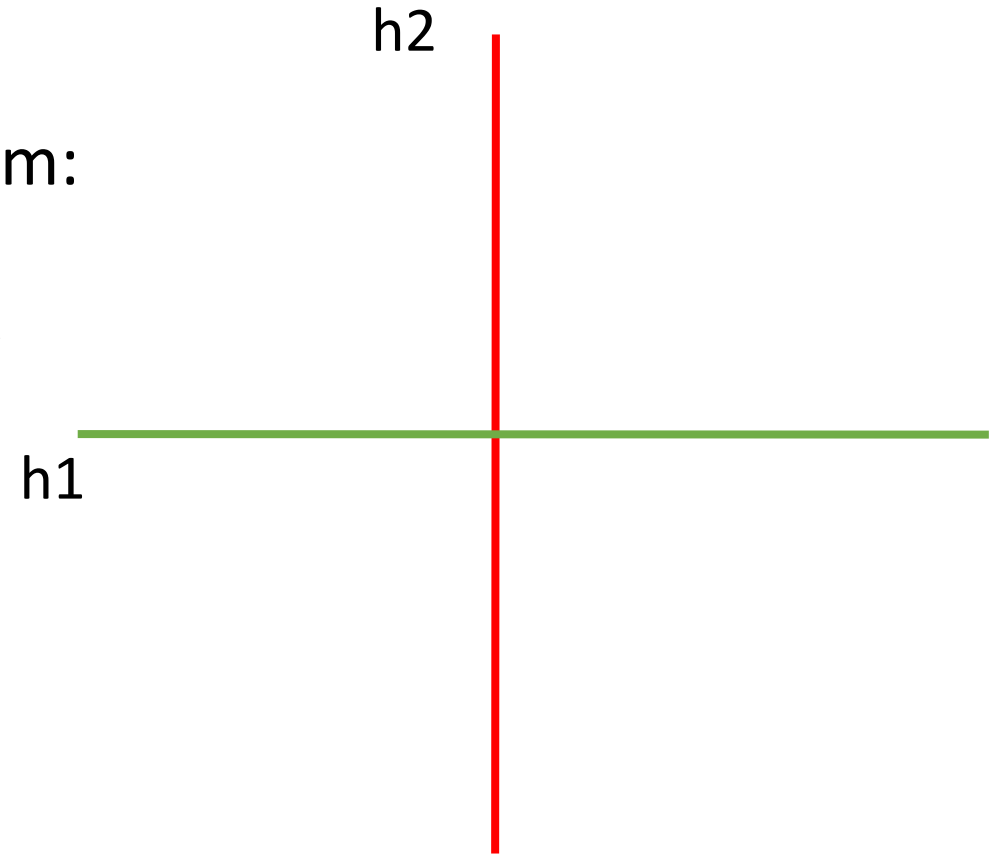
Consider a linear transform: h = Wx

Where x, h are both 2-dimensional



Feature transform:

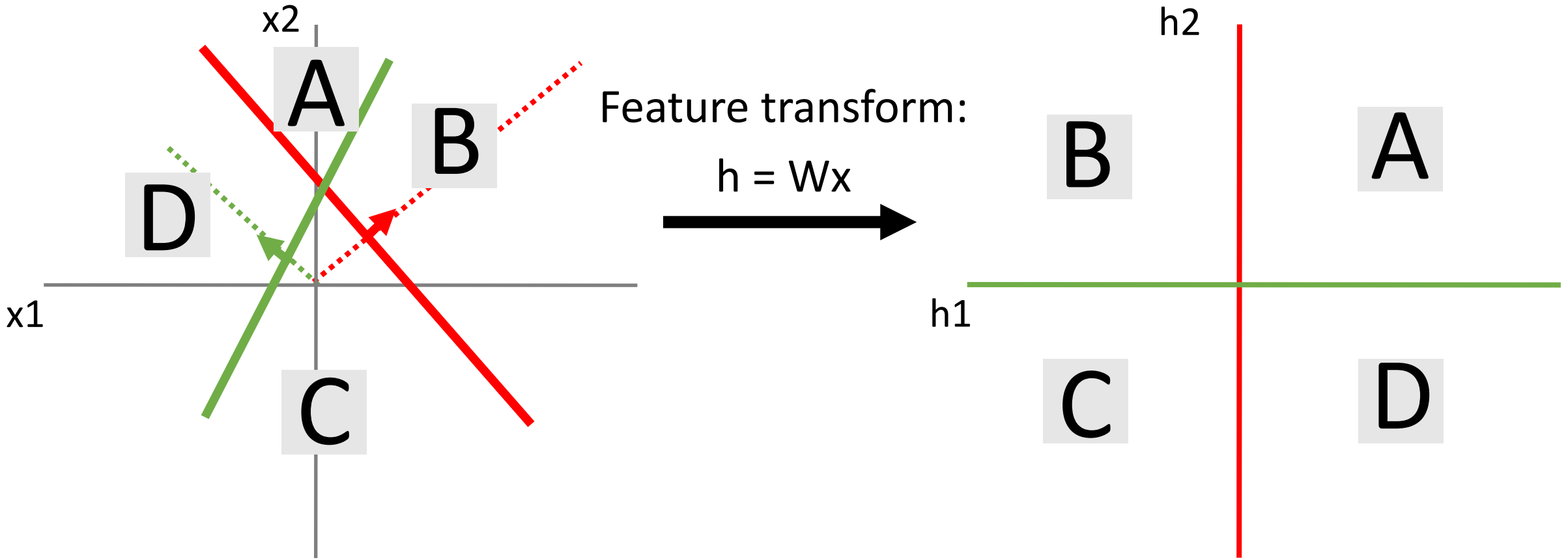
h = Wx



Space Warping

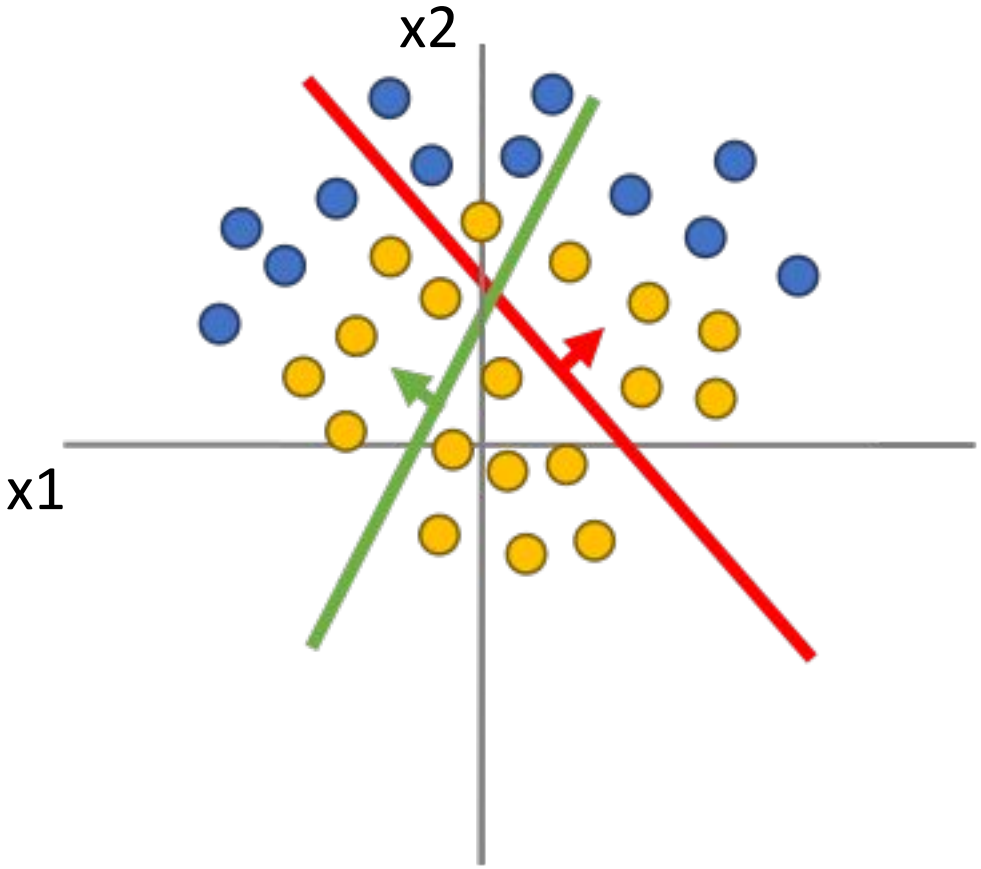
Consider a linear transform: h = Wx

Where x, h are both 2-dimensional



Space Warping

Points not linearly separable in original space



Consider a linear transform: h = Wx

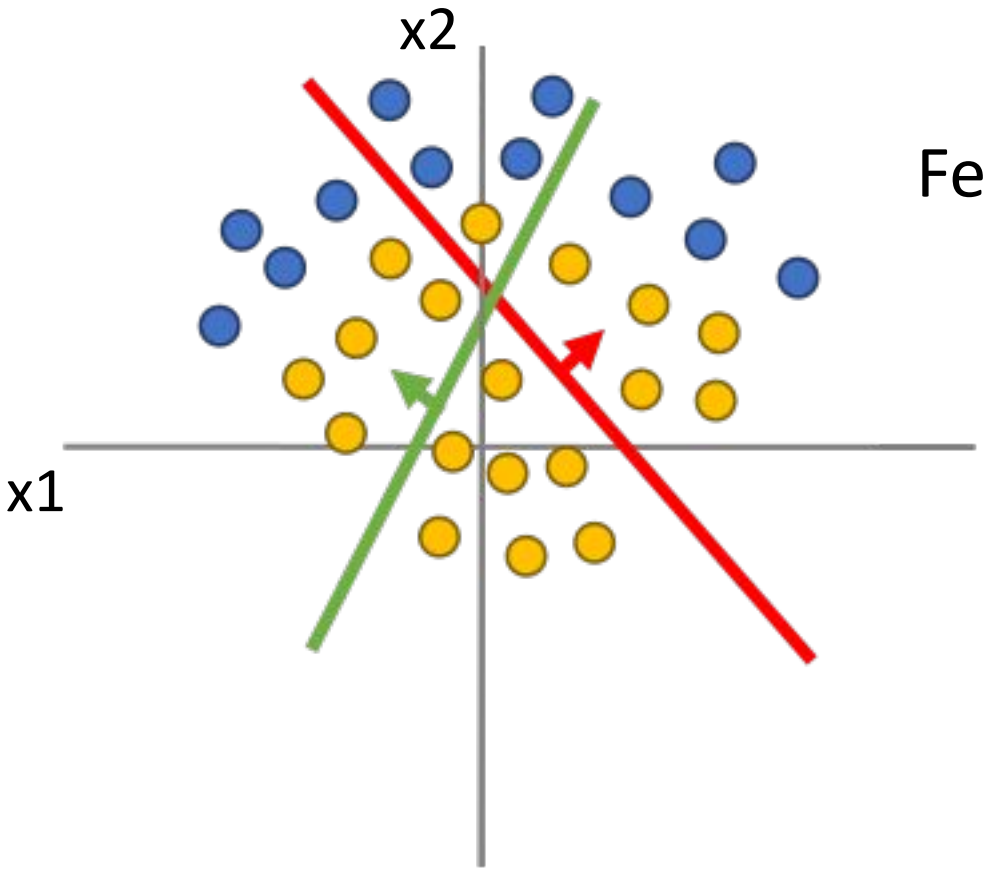
Where x, h are both 2-dimensional

Space Warping

Consider a linear transform: h = Wx

Where x, h are both 2-dimensional

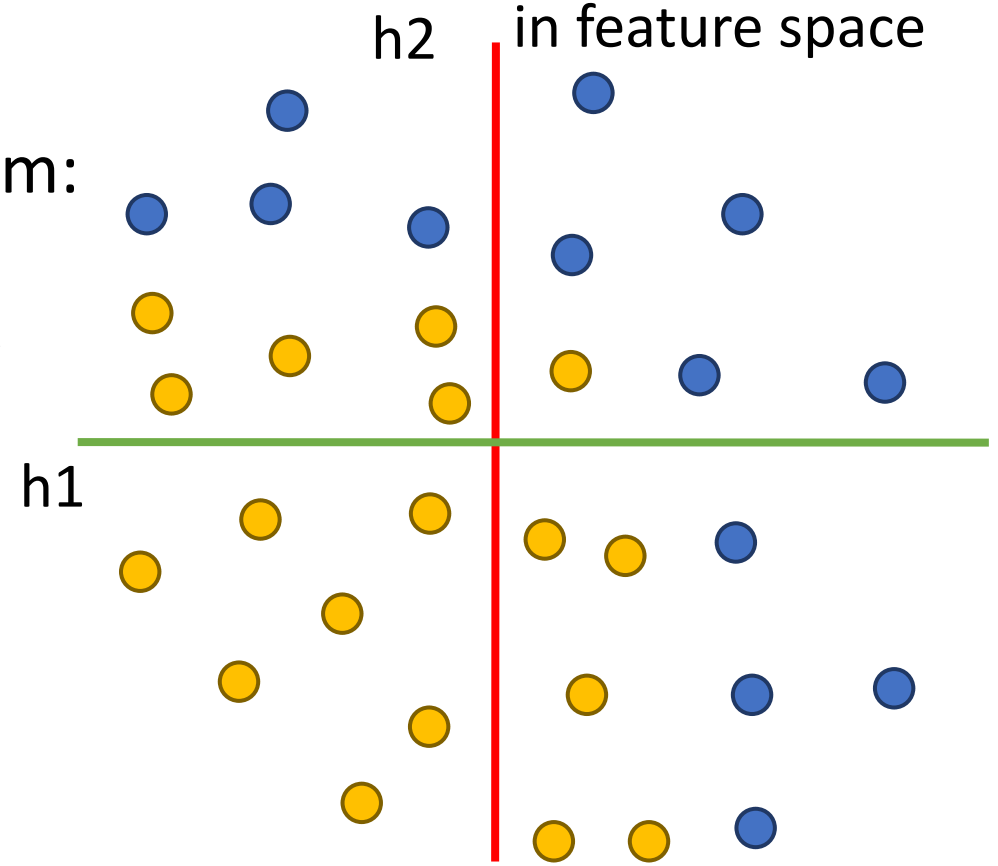
Points not linearly separable in original space



Feature transform:

h = Wx

Not linearly separable in feature space

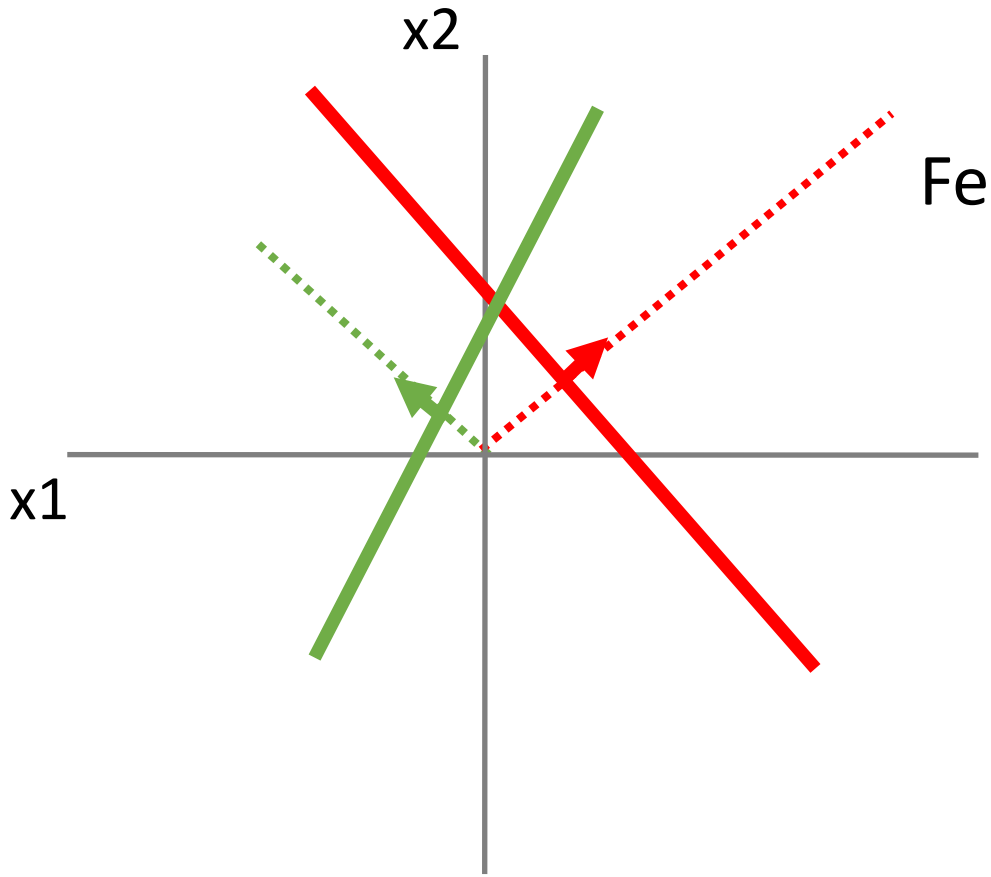


Space Warping

Consider a neural net hidden layer:

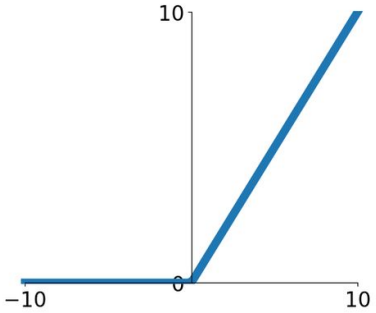
h = ReLU(Wx) = max(0, Wx)

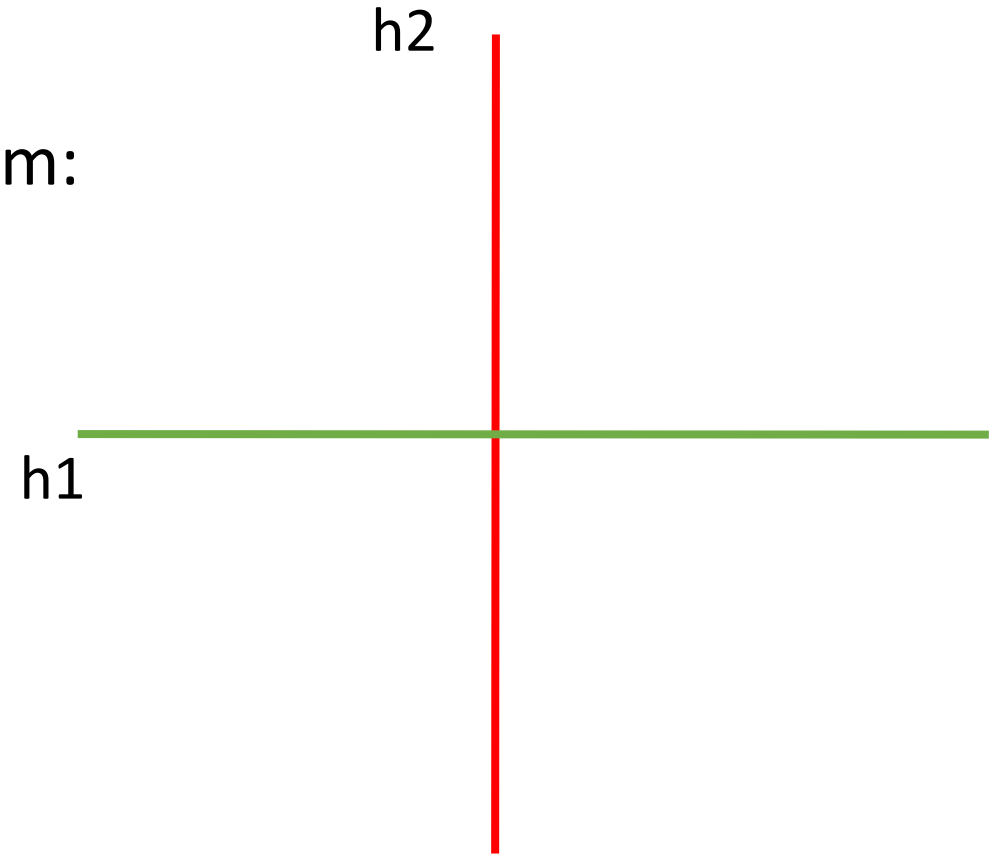
Where x, h are both 2-dimensional



Feature transform:

h = ReLU(Wx)



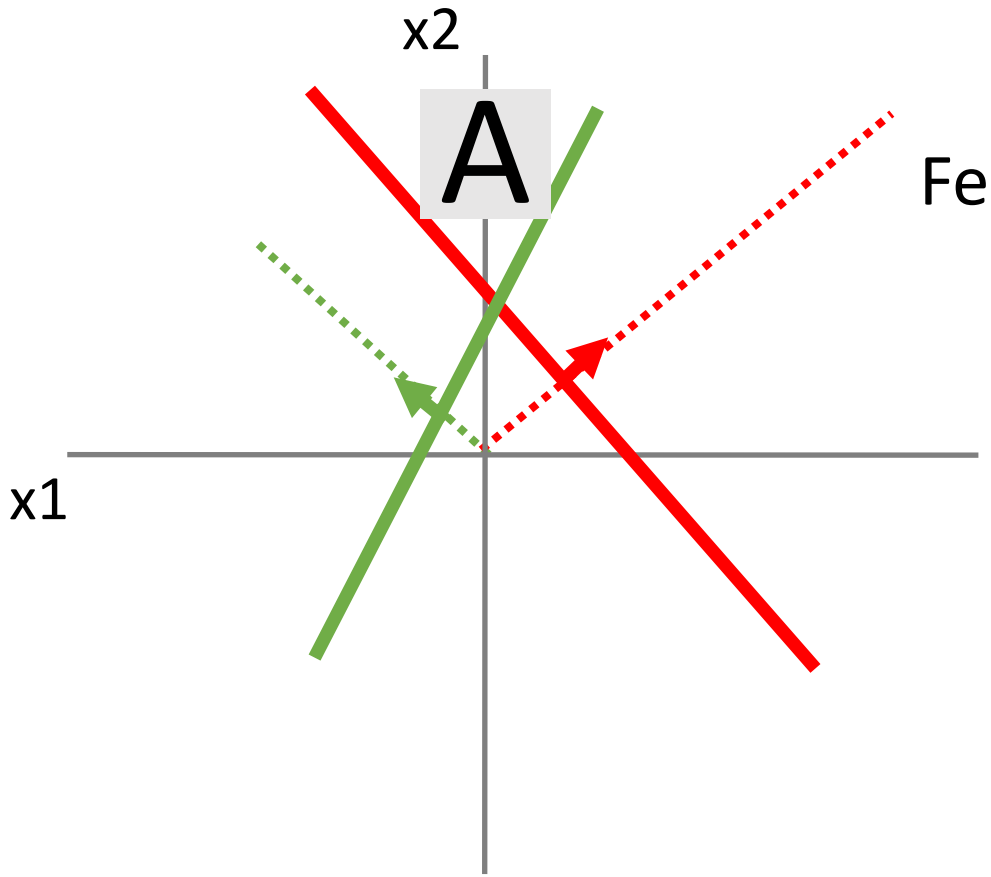


Space Warping

Consider a neural net hidden layer:

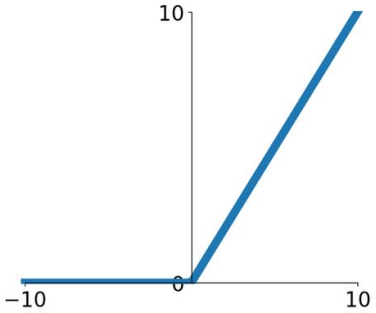
h = ReLU(Wx) = max(0, Wx)

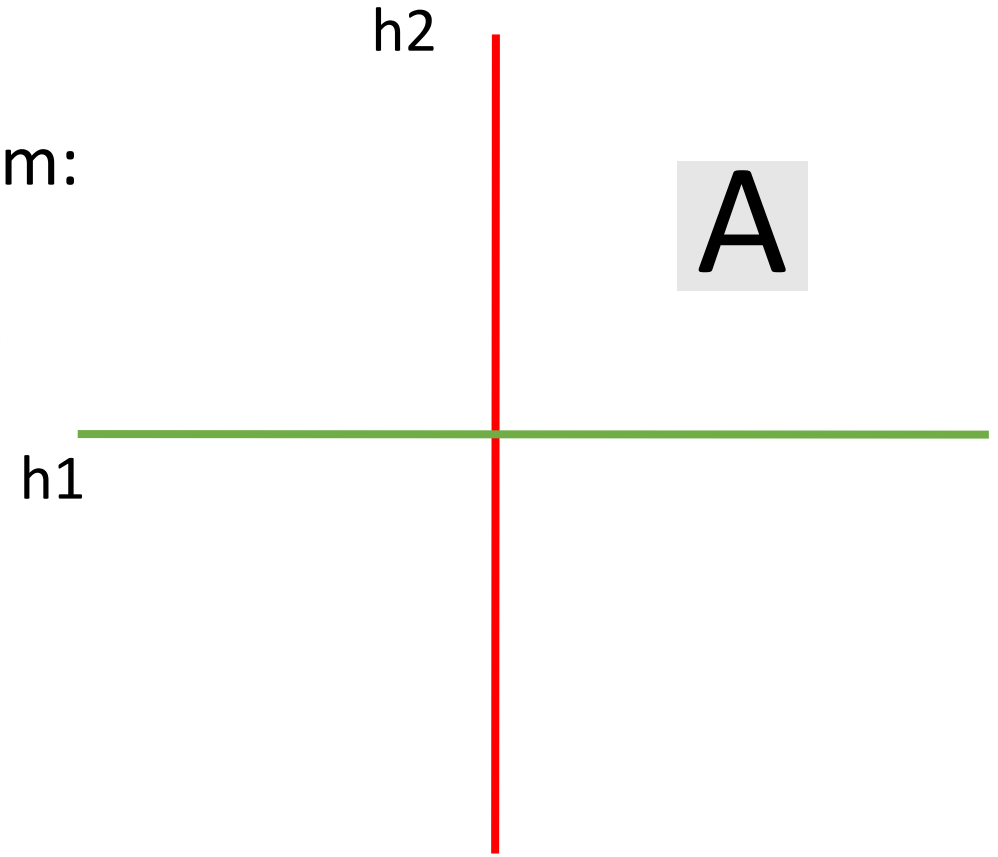
Where x, h are both 2-dimensional



Feature transform:

h = ReLU(Wx)



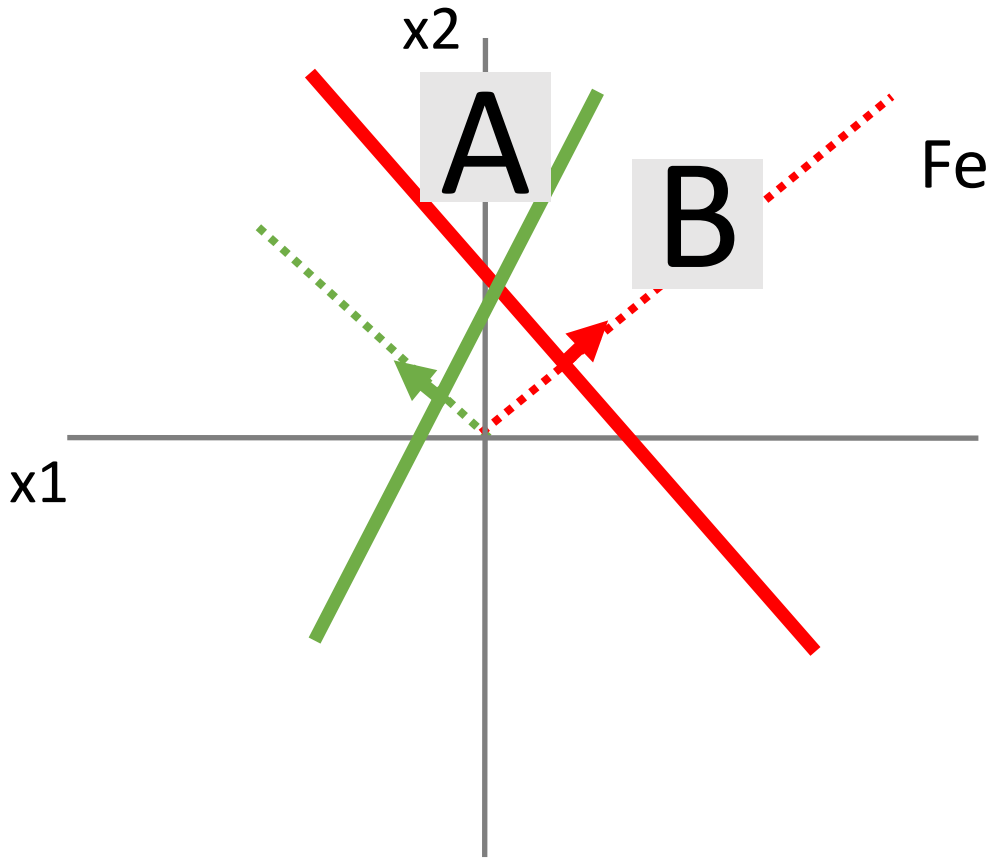


Space Warping

Consider a neural net hidden layer:

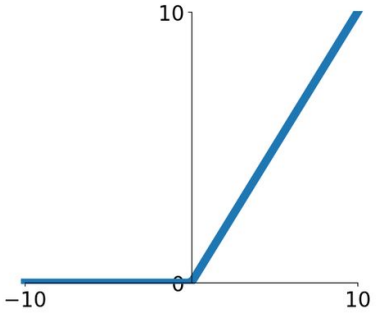
h = ReLU(Wx) = max(0, Wx)

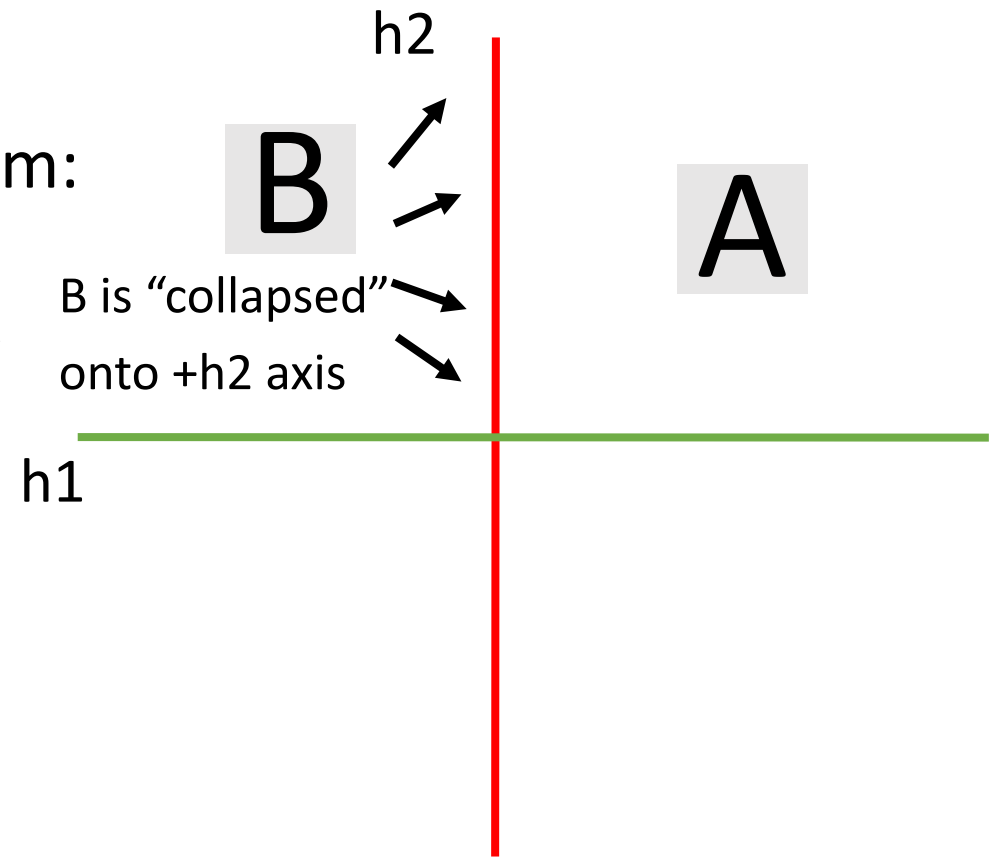
Where x, h are both 2-dimensional



Feature transform:

h = ReLU(Wx)



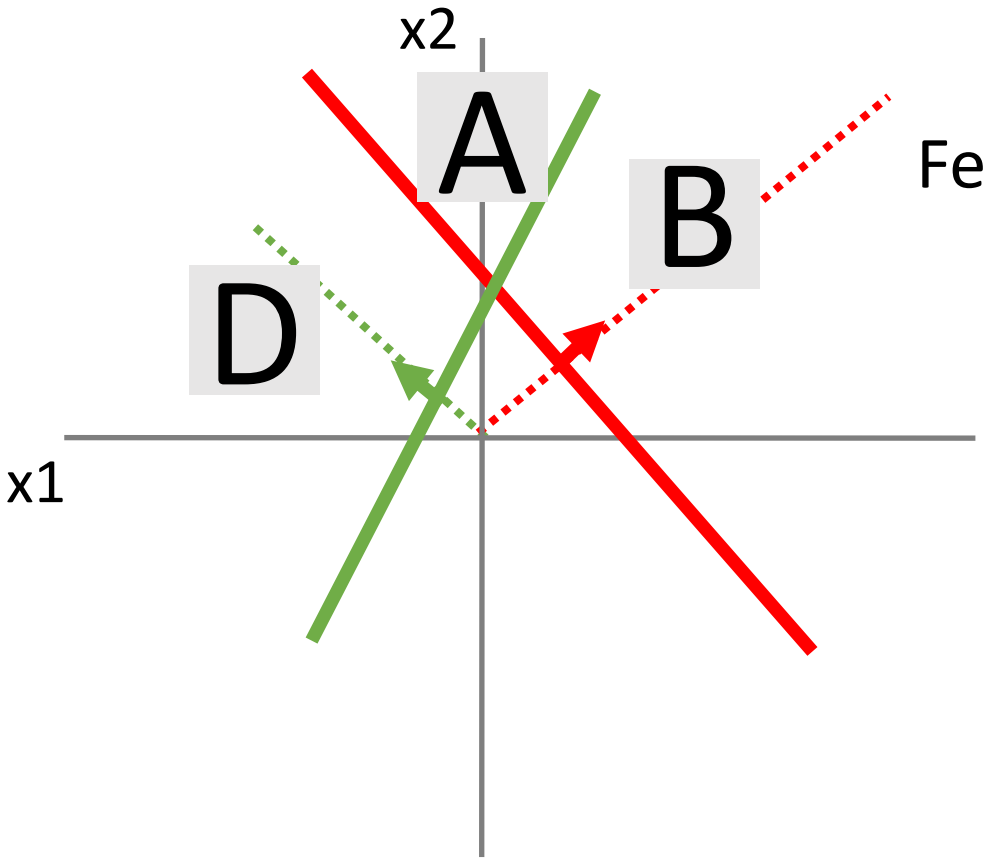


Space Warping

Consider a neural net hidden layer:

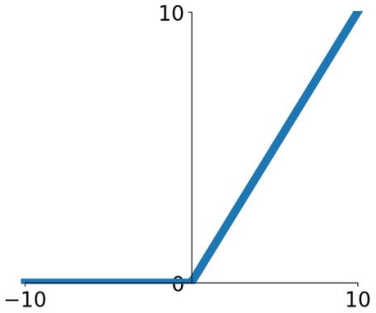
h = ReLU(Wx) = max(0, Wx)

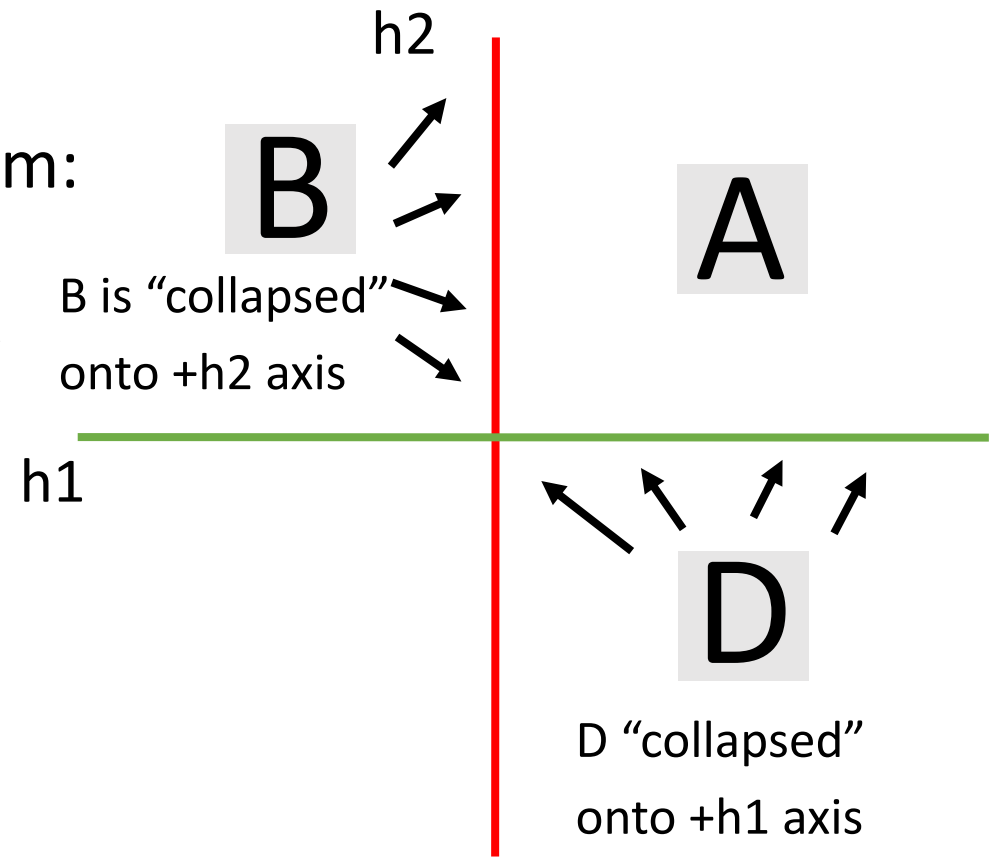
Where x, h are both 2-dimensional



Feature transform:

h = ReLU(Wx)



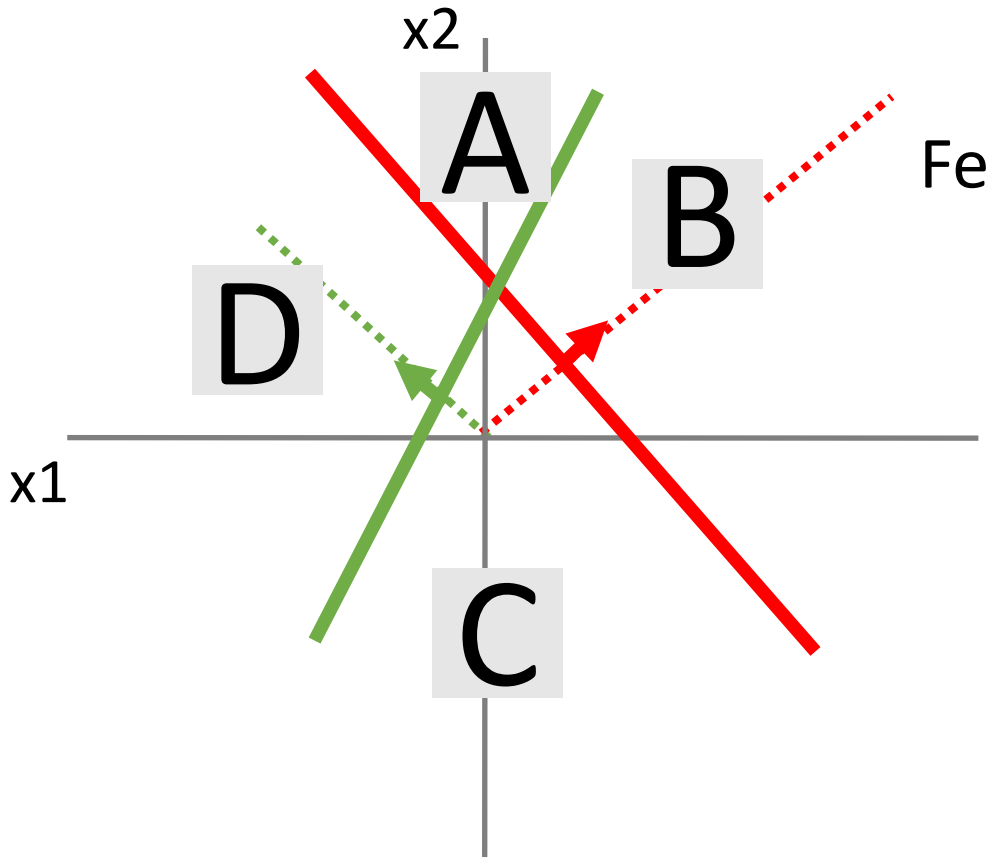


Space Warping

Consider a neural net hidden layer:

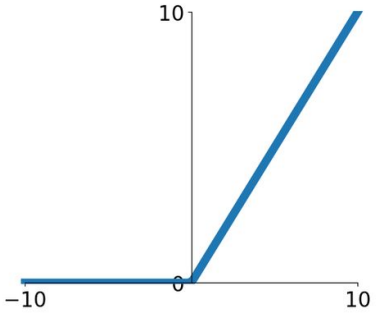
h = ReLU(Wx) = max(0, Wx)

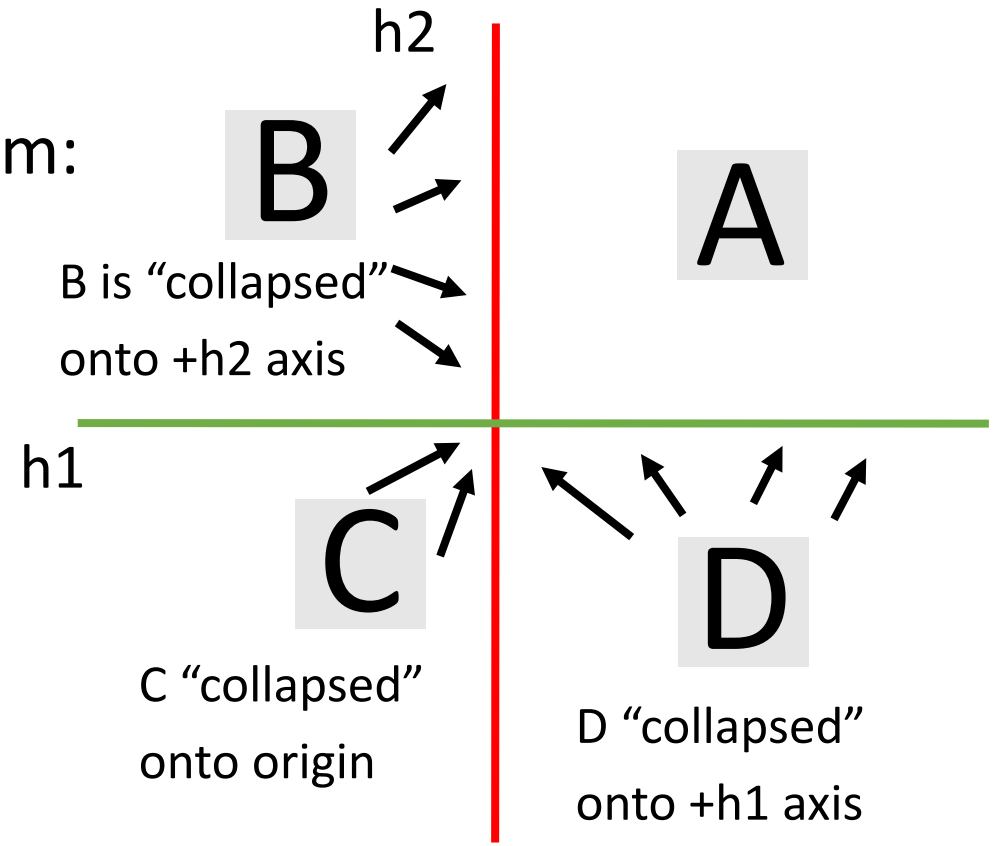
Where x, h are both 2-dimensional



Feature transform:

h = ReLU(Wx)





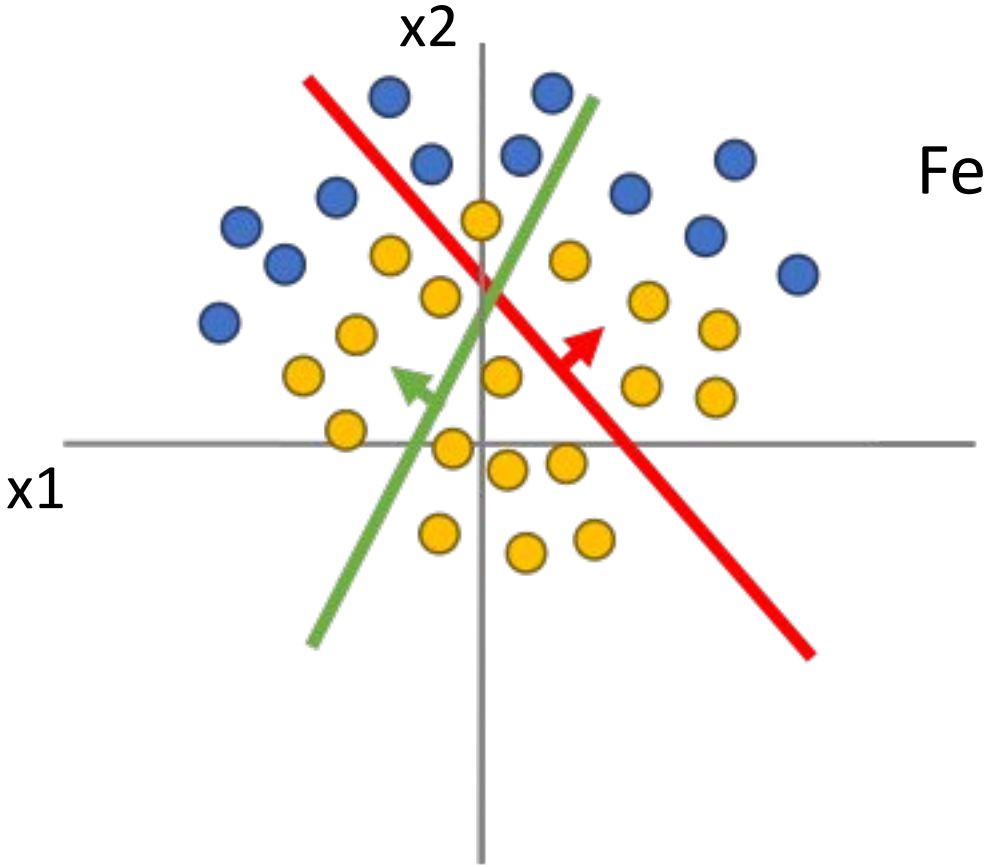
Space Warping

Consider a neural net hidden layer:

h = ReLU(Wx) = max(0, Wx)

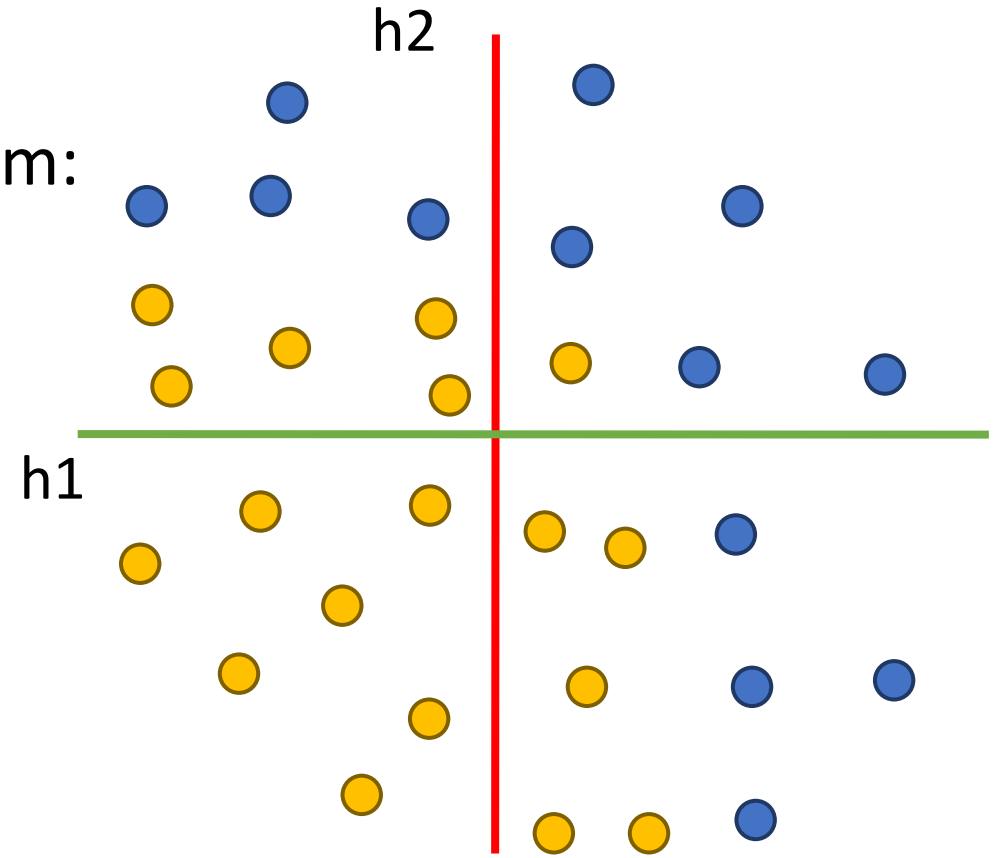
Where x, h are both 2-dimensional

Points not linearly separable in original space



Feature transform:

h = Wx



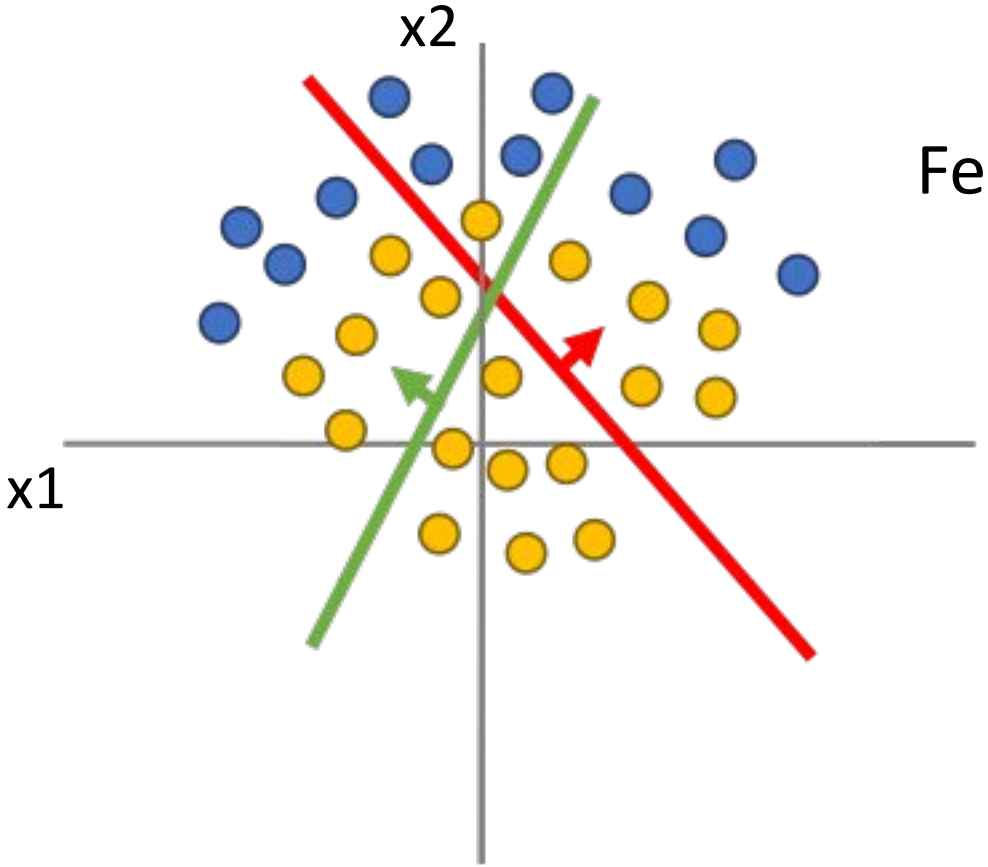
Space Warping

Consider a neural net hidden layer:

h = ReLU(Wx) = max(0, Wx)

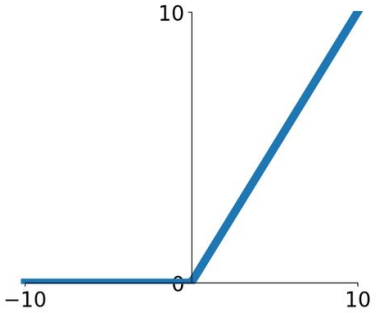
Where x, h are both 2-dimensional

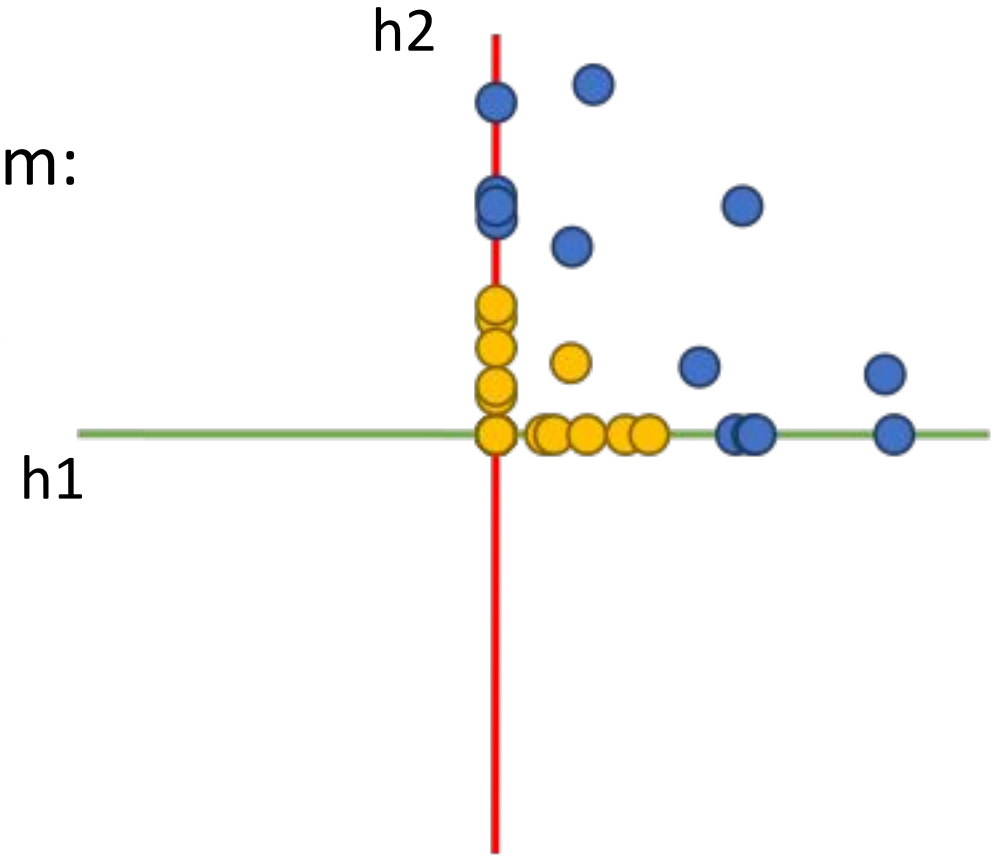
Points not linearly separable in original space



Feature transform:

h = ReLU(Wx)





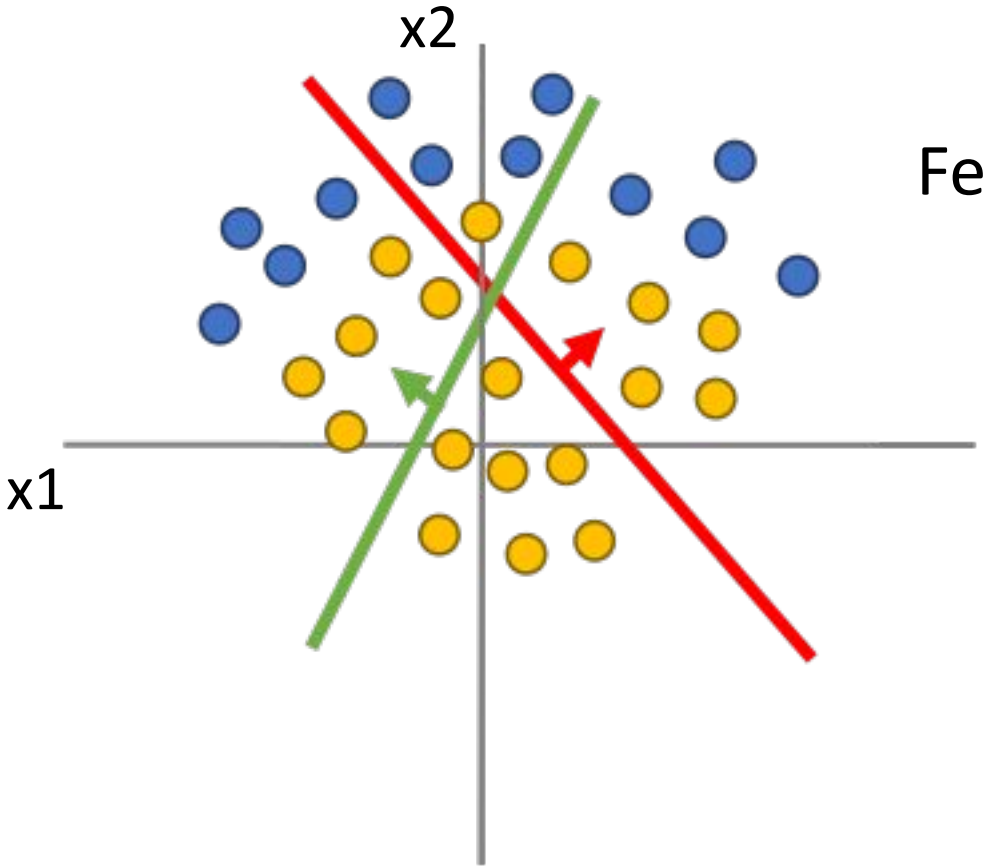
Space Warping

Consider a neural net hidden layer:

h = ReLU(Wx) = max(0, Wx)

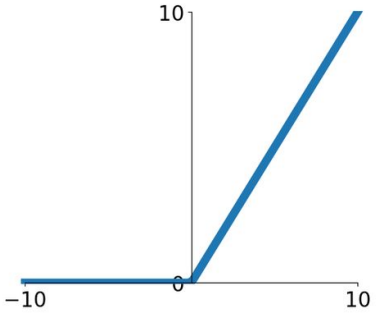
Where x, h are both 2-dimensional

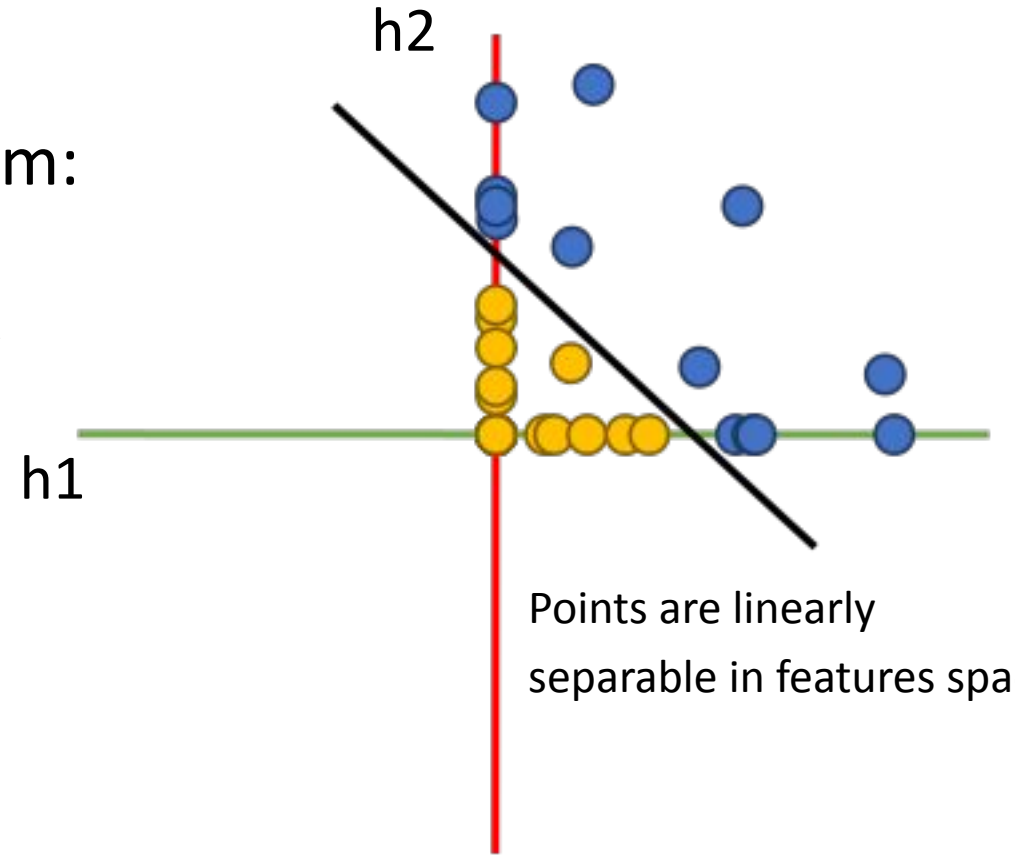
Points not linearly separable in original space



Feature transform:

h = ReLU(Wx)





Points are linearly separable in features space!

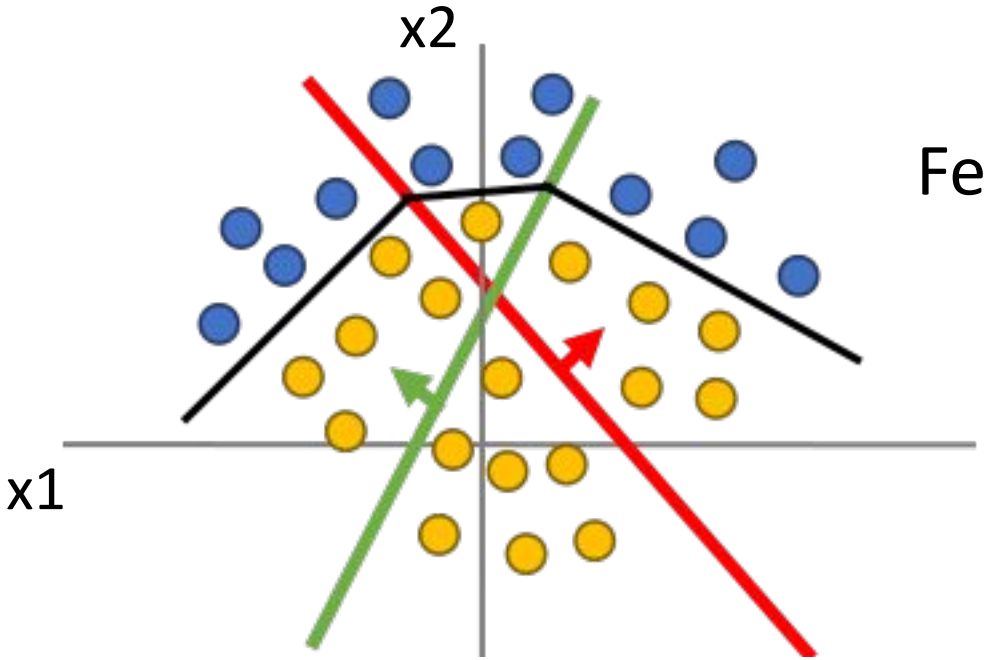
Space Warping

Consider a neural net hidden layer:

h = ReLU(Wx) = max(0, Wx)

Where x, h are both 2-dimensional

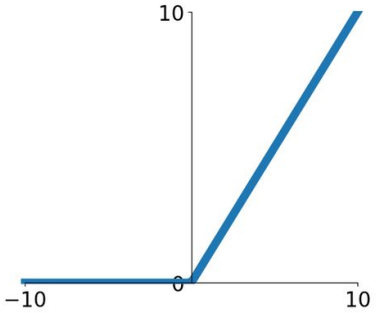
Points not linearly separable in original space

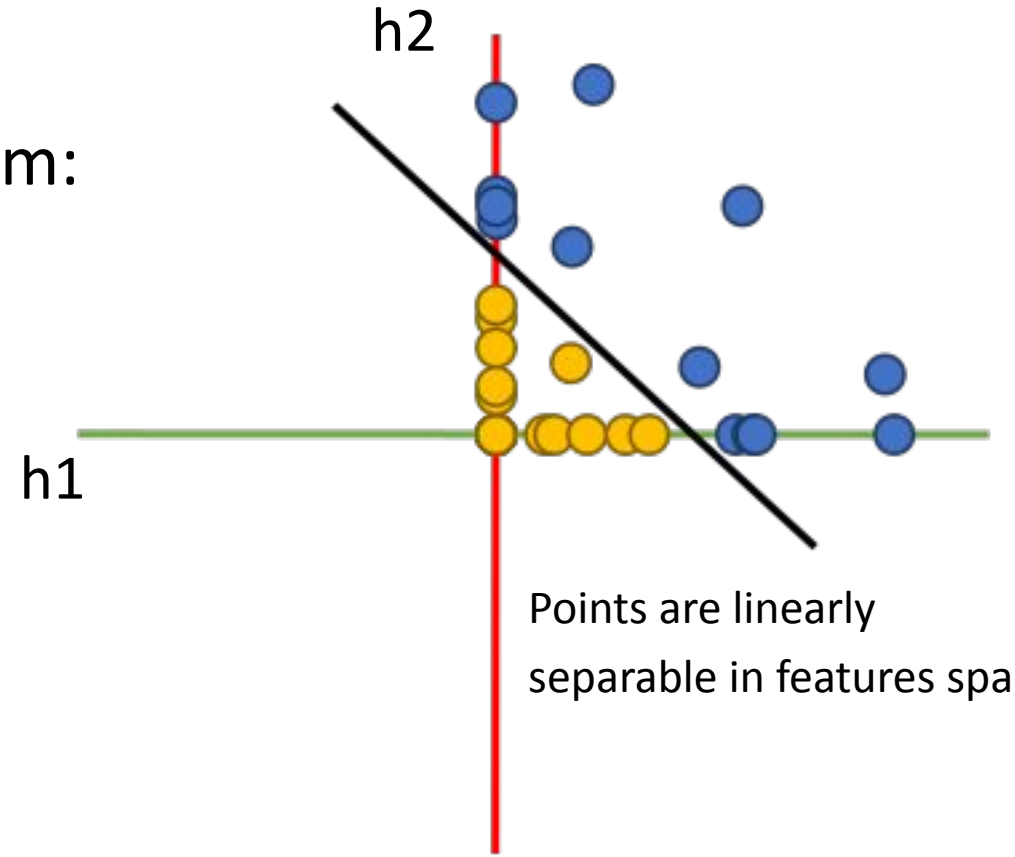


Linear classifier in feature space gives nonlinear classifier in original space

Feature transform:

h = ReLU(Wx)

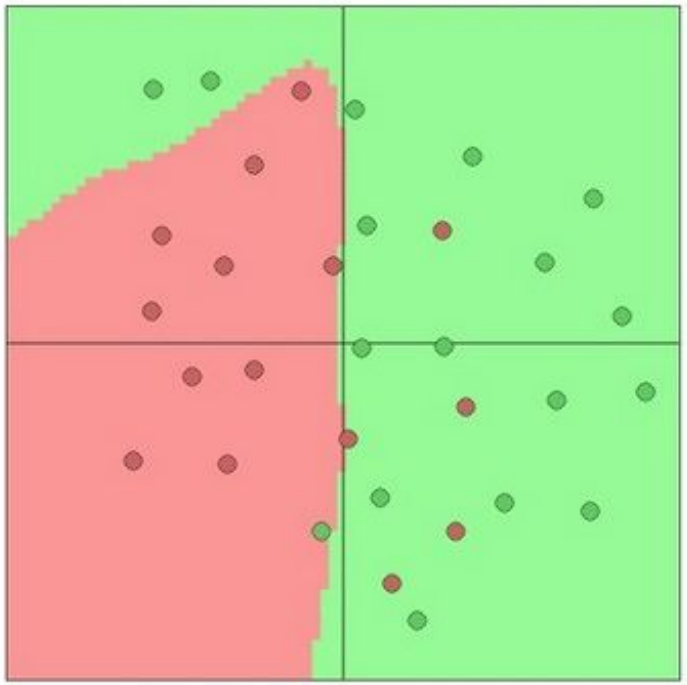




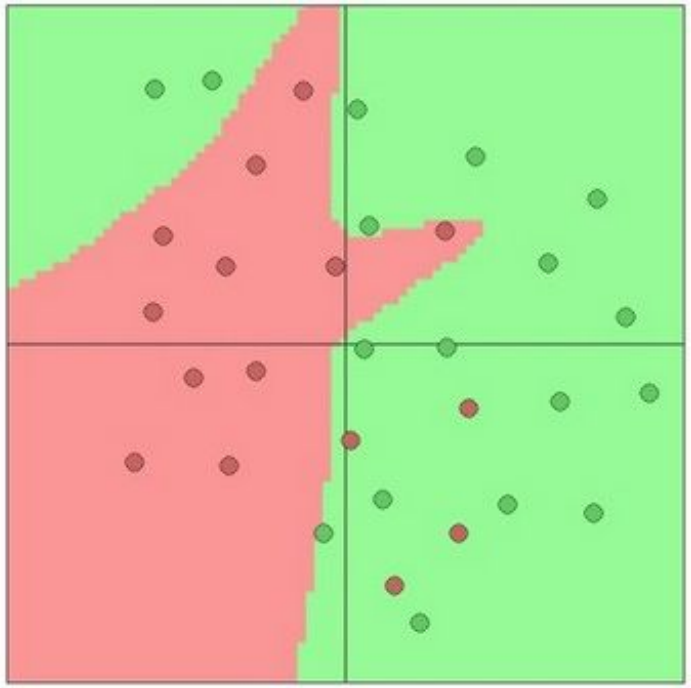
Points are linearly separable in features space!

Network width and capacity

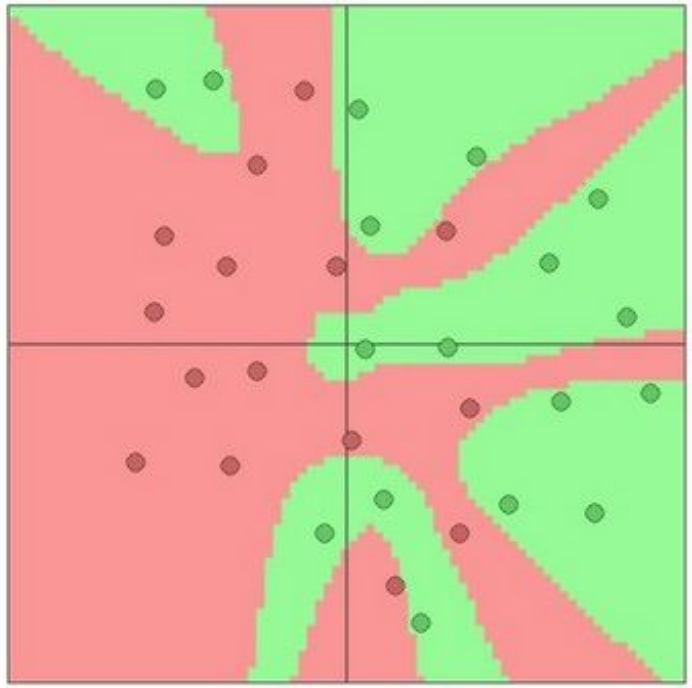
3 hidden units



6 hidden units

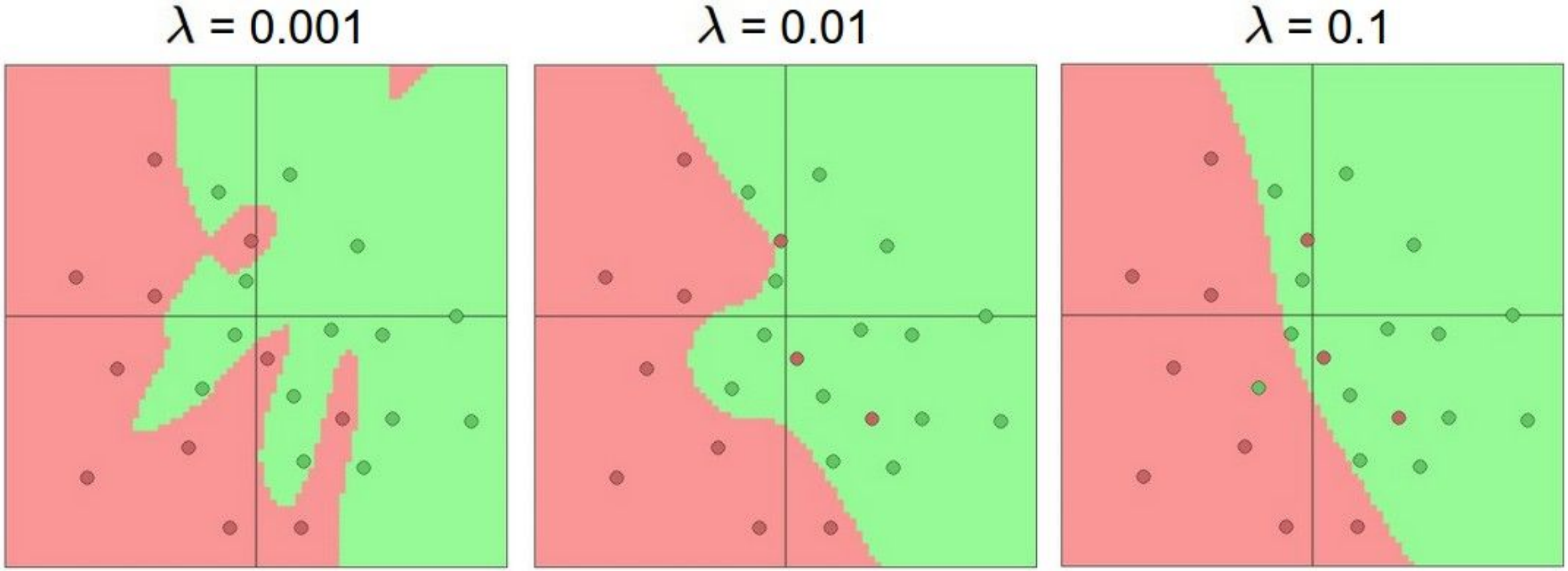


20 hidden units



More hidden units = more capacity

Capacity and regularization work together

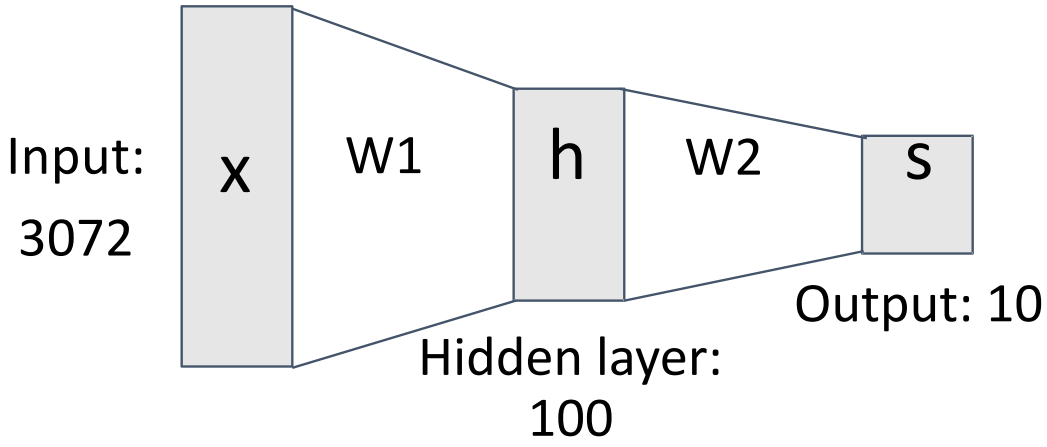


A wider network can represent a more complex boundary. Regularization controls how strongly the model uses that capacity.

Summary

From linear classifiers to fully-connected networks

$$f = W_2 \max(0, W_1 x)$$



Linear classifier: One template per class



Neural networks: Many reusable templates



Training requires gradients

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Example data loss: SVM

$$R(W) = \sum_k W_k^2$$

Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$

Total loss

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ we can learn W_1 and W_2

Test 1 :
https://docs.google.com/forms/d/e/1FAIpQLSf2RrN7hydXlxY0iPK6sbkk0yohaTyJHUMGRBJVla46Ad7A0Q/viewform?usp=dialog

Next lecture:
Backpropagation